

Proyecto Integrador

Ingeniería de Sistemas

Facultad de Ciencias Exactas - UNICEN



Visualización Geoposicionada y Categorización de Eventos

Alumno: Negretti Nahuel

Director: Dr. Luis Berdun

Co-Director: Dr. Marcelo Armentano

Índice

Índice.....	2
1. Introducción.....	5
2. Problema y Solución Propuesta.....	7
2.1 Captura y Análisis de Requerimientos.....	7
2.2 Esquema General de la Solución.....	8
2.3 Adquisición de Datos.....	9
2.4 Procesamiento Inteligente.....	10
2.4.1 Clasificación de Noticias.....	10
2.4.1.1 Aprendizaje Supervisado vs. No Supervisado.....	10
2.4.1.2 Construcción del Dataset.....	10
2.4.1.3 Procesamiento del Texto y NLP.....	11
2.4.1.4 Entrenamiento del Modelo Clasificador.....	11
2.4.2 Named Entity Recognition (NER).....	12
2.4.2.1 Evaluación del Uso de SpaCy.....	12
2.4.2.2 Método Basado en Reglas para Extracción de Ubicaciones.....	12
2.4.2.3 Estrategia de Geolocalización y Normalización de Direcciones.....	13
2.4.2.4 Integración con la Base de Datos.....	13
2.5 Repositorio de Noticias.....	14
2.5.1 Diseño de la Base de Datos.....	14
2.5.2 Aplicación de la Normalización.....	16
2.6 Exposición y Visualización de Datos.....	17
2.6.1 Funcionamiento del Backend.....	17
2.6.2 Mecanismos de Filtrado de Noticias.....	17
2.6.3 Comunicación entre la Aplicación Web y el Backend.....	17
2.6.4 Interfaz Web y Experiencia de Usuario.....	18
3. Implementación.....	21
3.1 Estructura General del Sistema.....	21
3.2 Adquisición de datos.....	22
3.3 Clasificación Automática del Tipo de Noticia.....	24
3.3.1 Evaluación de Técnicas y Selección del Modelo.....	25
3.3.2 Comparación de Modelos y Vectorizadores.....	26
3.3.3 Desempeño del Modelo Seleccionado (SVM + TF-IDF).....	27
3.3.4 Análisis Cualitativo del Modelo Seleccionado.....	28
3.3.4.1 Matriz de Confusión Normalizada.....	28
3.3.4.2 Ejemplos de Errores de Clasificación.....	29
3.3.4.3 Palabras Clave por Clase.....	29
3.3.4 Integración del Modelo en el Sistema.....	30
3.4 Detección y Georreferenciación de Entidades Geográficas.....	31
3.4.1 Enfoque Implementado.....	31
3.4.2 Normalización de Nombres de Calles.....	31
3.4.3 Flujo de Procesamiento.....	32

3.4.4 Consulta a la API de Geocodificación.....	34
3.4.5 Extracción Final.....	34
3.4.6 Dificultades Encontradas.....	35
3.4.7 Resultados Esperados.....	35
3.5 Automatización con Cron en Contenedor.....	36
3.5.1 Estructura y Configuración.....	36
3.5.2 Programación de Tareas.....	37
3.5.3 Manejo de Variables de Entorno.....	37
3.5.4 Dependencias Utilizadas.....	38
3.6 Repositorio de Noticias.....	38
3.6.1 Configuración de Conexión en settings.py.....	38
3.6.2 Estructura de los Modelos en Django.....	38
3.6.3 Flujo de Almacenamiento.....	39
3.6.4 Consideraciones Adicionales.....	40
3.6.5 Ventajas del Uso de Django.....	41
3.7 Exposición de Datos (Backend del Sistema).....	41
3.7.1 Arquitectura General.....	41
3.7.2 Modelos y Persistencia de Datos.....	41
3.7.3 Serializadores.....	42
3.7.4 Endpoints y Lógica de Almacenamiento.....	42
3.7.5 Configuración y Despliegue con Docker.....	43
3.8 Visualización de Datos (Frontend del Sistema).....	44
3.8.1 Estructura del Proyecto.....	44
3.8.2 Componentes Principales.....	44
3.8.3 Consumo de la API del Backend.....	45
3.8.4 Variables de Entorno en Next.js.....	46
3.8.5 Despliegue con Docker.....	46
3.8.6 Estilo Visual.....	47
3.8.7 Optimización del Renderizado.....	49
4. Casos de Estudio.....	50
4.1 Caso de Estudio 1: Accidente Vial Detectado Correctamente y Procesado con Éxito.. 50	
4.1.1 Fuente de Datos.....	50
4.1.2 Noticia Extraída.....	50
4.1.3 Clasificación Automática.....	51
4.1.4 Extracción de Entidades y Geolocalización.....	51
4.1.5 Resultado Final.....	52
4.1.6 Análisis.....	52
4.2 Caso de Estudio 2: Clasificación Incorrecta de un Caso de Abuso como "Asesinato".. 52	
4.2.1 Detalles del Caso.....	52
4.2.2 Análisis del Error.....	52
4.2.3 Conclusión del Caso.....	53
4.3 Caso de Estudio 3: Falla en la Geolocalización por Falta de Normalización de Entidad.....	53

4.3.1 Detalles del Caso.....	53
4.3.2 Problema Detectado.....	54
5.3.3 Solución Propuesta.....	54
4.3.4 Conclusión del Caso.....	54
5. Conclusión.....	56
5.1 Desarrollo del Trabajo.....	56
5.2 Limitaciones.....	57
5.3 Trabajos Futuros.....	57
6. Referencias.....	59

1. Introducción

En un mundo cada vez más digitalizado, la disponibilidad de información se ha multiplicado exponencialmente. Las noticias sobre eventos urbanos —como accidentes viales, delitos o incendios— se publican constantemente en medios digitales y redes sociales, generando un flujo continuo de datos difíciles de procesar manualmente por el usuario común. Esta sobrecarga informativa plantea un problema claro: ¿cómo acceder de forma eficiente y personalizada a los eventos que realmente importan, ya sea por su cercanía geográfica o por su tipo?

La Ingeniería en Sistemas, como disciplina, ofrece herramientas para abordar este tipo de problemas complejos mediante el diseño y desarrollo de soluciones informáticas que integren procesamiento inteligente de datos, automatización y experiencia de usuario. Este proyecto surge como respuesta a dicha necesidad, proponiendo el desarrollo de un sistema que automatice la recolección, clasificación, geolocalización y visualización de noticias locales, priorizándolas según los intereses y ubicación del usuario.

A modo de ejemplo, considérese una situación hipotética donde existen cuatro noticias recientes:

1. Un accidente vial en Londres.
2. Un robo en la Facultad de Ciencias Exactas en Tandil.
3. Un accidente vial en la ruta 226, a la altura de Azul.
4. Un robo en Av. Buzón y Sarmiento.

Si el usuario se encuentra ubicado en la intersección de Pinto y 9 de Julio, el sistema debe priorizar los eventos de acuerdo con la proximidad geográfica. En este caso, el orden sería: 4, 2, 3 y 1. No obstante, si el usuario está más interesado en accidentes viales, el orden de prioridad cambiaría a 3, 1, 4 y 2. E incluso, si filtrara solo por tipo de evento, el sistema mostraría exclusivamente las noticias 3 y 1. Este simple ejemplo ilustra la necesidad de combinar dos ejes fundamentales: la **temática del evento** y su **cercanía relativa al usuario**.

A lo largo del desarrollo de este proyecto, se integraron múltiples tecnologías y conceptos abordados en la carrera: desde web scraping —una técnica automatizada para extraer datos estructurados desde páginas web, ampliamente utilizada para recolectar información pública en contextos donde no se dispone de una API formal ([Glez-Peña et al., 2014](#)) — con Scrapy ([Scrapy, 2024](#)), procesamiento de lenguaje natural (NLP) y aprendizaje automático supervisado ([Bishop, 2006](#)) para la clasificación, hasta la implementación de un sistema web basado en Django (backend) ([Django Software Foundation, 2025](#)) y Next.js (frontend) ([Vercel, 2025](#)), todo orquestado mediante contenedores Docker ([Docker Inc, 2025](#)) para asegurar modularidad y facilidad de despliegue.

El valor diferencial del sistema no reside solo en su capacidad de categorizar noticias en tiempo real y mapearlas geográficamente, sino también en su diseño escalable y adaptable. Una arquitectura modular, basada en la separación de responsabilidades y el uso de componentes desacoplados, junto con buenas prácticas de ingeniería de software, permite que la solución evolucione fácilmente y pueda ser adaptada a otros contextos.

Este informe describe en detalle todas las etapas del proyecto: desde la captura de requerimientos ([Capítulo 2](#)) hasta la implementación técnica ([Capítulo 3](#)), incluyendo decisiones de diseño, tecnologías utilizadas y análisis de resultados. Asimismo, se presentan casos de estudio reales que ilustran el funcionamiento del sistema ([Capítulo 4](#)), así como una reflexión crítica sobre las limitaciones encontradas y posibles líneas de mejora ([Capítulo 5](#)). La experiencia no solo representó un ejercicio de aplicación práctica, sino también un espacio de síntesis de los aprendizajes adquiridos a lo largo de la carrera.

2. Problema y Solución Propuesta

La difusión masiva de noticias digitales ha generado un nuevo desafío para los usuarios: acceder de forma rápida y ordenada a aquella información que les resulta verdaderamente relevante. En particular, los eventos de tipo policial —como robos, accidentes, incendios o hechos violentos— pueden tener un impacto directo en la vida de los ciudadanos, pero muchas veces se encuentran dispersos entre múltiples portales, sin una estructura común ni una jerarquización clara.

Actualmente, la mayoría de los medios de noticias no ofrece mecanismos de personalización según la ubicación o intereses del lector. Tampoco clasifican los artículos con etiquetas estructuradas, lo que obliga al usuario a leer titulares o textos completos para determinar si una noticia le concierne. Además, muchas de las ubicaciones mencionadas en las noticias están expresadas en lenguaje coloquial o informal (“Cabral y Mitre”, “cerca de Puente Azul”), lo que dificulta su análisis automático y su representación geográfica precisa.

Esta falta de priorización, estructura y contextualización espacial genera una sobrecarga informativa que afecta la toma de decisiones, especialmente para usuarios que desean conocer qué ocurre cerca de ellos o en relación con temáticas específicas como delitos o accidentes.

El objetivo del proyecto es abordar esta problemática mediante el desarrollo de un sistema que automatice el proceso de recolección, clasificación y geolocalización de noticias, ofreciendo al usuario una interfaz clara, filtrable y priorizada según su ubicación e intereses. De esta forma, se busca mejorar el acceso a información urbana relevante y promover un uso más eficiente de los datos disponibles.

2.1 Captura y Análisis de Requerimientos

Para el diseño del sistema, se tomaron en cuenta los conceptos abordados en la materia Metodología de Desarrollo de Software, aplicando principios de **ingeniería de software** orientados a la captura y análisis de requerimientos. Esto permitió definir una solución alineada con las necesidades del usuario final y garantizar un desarrollo estructurado.

Como punto de partida, se realizó un estudio exploratorio sobre el consumo de noticias en entornos digitales, identificando necesidades clave relacionadas con la **priorización de información**, la **clasificación automática de noticias** y la **geolocalización de eventos**. Estos hallazgos guiaron la definición de los requerimientos del sistema.

Durante esta fase, se consideraron los siguientes aspectos:

- **Requerimientos funcionales:** Se establecieron las funcionalidades esenciales del sistema, incluyendo la recolección de noticias mediante *web scraping*, la clasificación automática de eventos y la geolocalización de información relevante.
- **Requerimientos no funcionales:** Se definieron atributos de calidad, priorizando la **escalabilidad**, la **eficiencia en el procesamiento de datos** y la **accesibilidad** desde distintos dispositivos, con el fin de mejorar la experiencia del usuario.

- **Modelo iterativo de desarrollo:** Si bien no se adoptó una metodología rígida, se siguió un enfoque iterativo, permitiendo realizar ajustes progresivos a medida que se evaluaban los resultados en cada fase del proceso.

El análisis de estos requerimientos permitió establecer una organización del sistema basada en una **arquitectura modular**, en la cual cada componente cumple una función específica dentro del flujo de procesamiento de datos. Esta estructura favorece la separación de responsabilidades, optimiza el rendimiento general y facilita la mantenibilidad y escalabilidad del software, permitiendo además una evolución flexible del sistema a medida que se incorporen nuevas funcionalidades.

2.2 Esquema General de la Solución

La **Figura 1** representa el recorrido completo de los datos, desde su origen hasta su presentación final. El sistema se compone de diferentes etapas, cada una con un propósito específico. La solución contempla dos grandes componentes: un sistema de procesamiento de datos (backend) y una interfaz de usuario (frontend) encargada de mostrar la información procesada.

A continuación, se describen las principales etapas que conforman el flujo de datos dentro del sistema:

- **Adquisición de datos (Procesamiento Intermedio):** Se recolecta información desde múltiples portales de noticias mediante técnicas de web scraping. Luego, los datos obtenidos son estructurados y preparados para su posterior análisis.
- **Procesamiento Inteligente:** Una vez recolectadas las noticias, se aplican diversas técnicas de procesamiento automatizado para analizar y extraer información estructurada de su contenido. Esta etapa se divide en dos procesos principales.
 - **Clasificación automática del tipo de noticia:** Las noticias son analizadas mediante modelos de clasificación que identifican su categoría según el contenido textual. Las principales clases definidas incluyen “Accidente”, “Robo”, “Incendio”, entre otras. Este proceso permite organizar la información y mejorar la experiencia de búsqueda del usuario final.
 - **Detección y georreferenciación de entidades geográficas:** Se aplican técnicas de reconocimiento de entidades nombradas (Named Entity Recognition – NER) para identificar lugares relevantes dentro del texto, como calles, barrios o ciudades. Luego, mediante procesos de geocodificación ([Zandbergen, 2008](#)), estos lugares son transformados en coordenadas geográficas (latitud y longitud), lo que permite ubicar las noticias en un mapa interactivo.
- **Repositorio de noticias (Almacenamiento en Base de Datos):** Una vez procesada, la información se almacena en una base de datos relacional, organizada de forma estructurada para permitir búsquedas eficientes y consultas por parte del sistema.
- **Exposición y Visualización de Datos:** La información almacenada es consumida por una aplicación que permite a los usuarios acceder a los eventos priorizados según sus preferencias. Esta aplicación cuenta con filtros por ubicación e intereses, y se actualiza dinámicamente mediante la comunicación bidireccional con el sistema de backend.

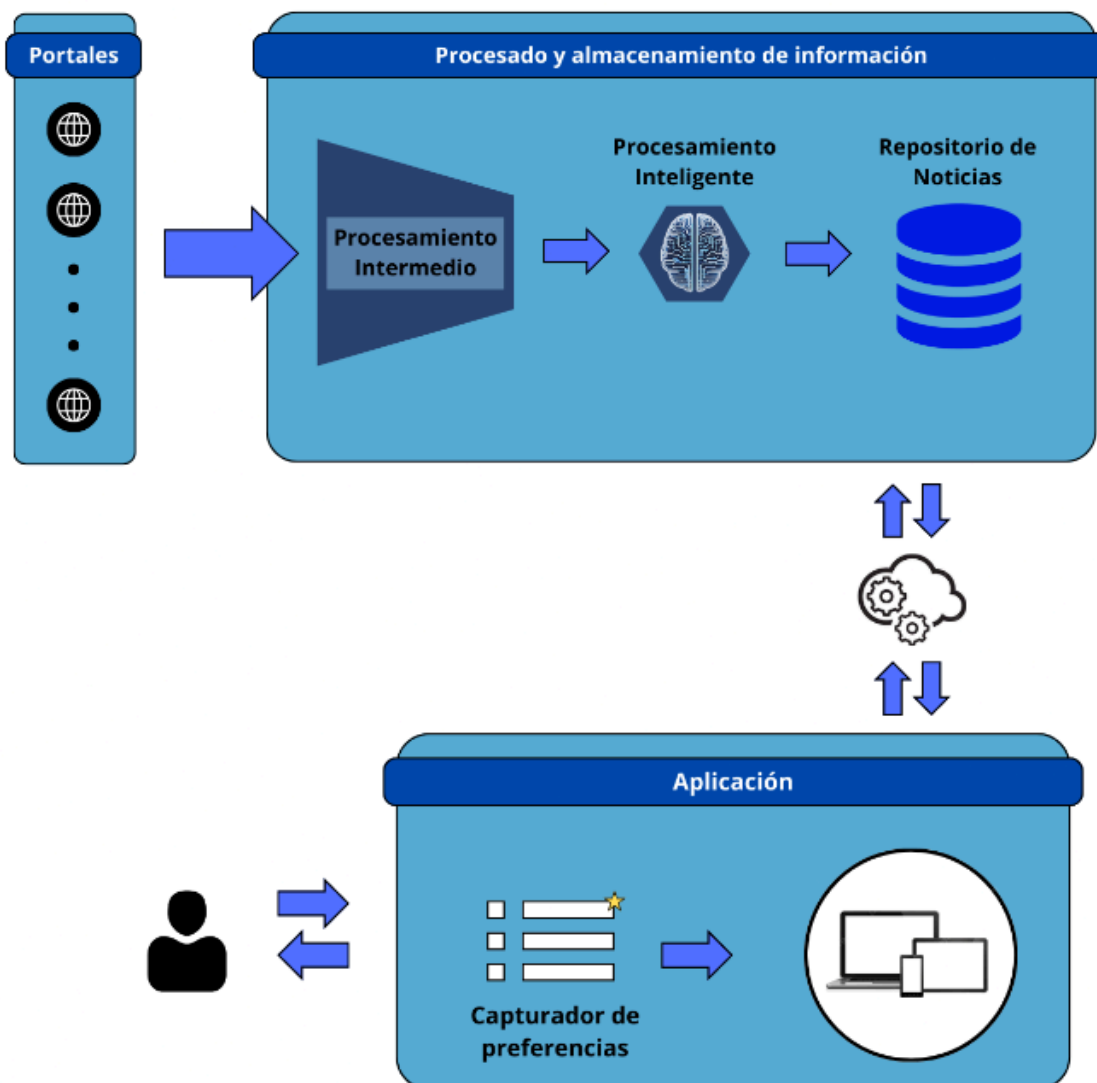


Figura 1. Esquema general de la solución propuesta

Esta solución retoma los principios de una **arquitectura modular** —concepto abordado en la materia Diseño de Sistemas de Software— y los aplica en una estructura clara y funcional que facilita la presentación de la información procesada.

2.3 Adquisición de Datos

En esta etapa se utilizó el framework **Scrapy** ([Scrapy, 2024](#)), una herramienta de *Python* especializada en *web scraping*, que permite extraer información de múltiples fuentes de manera eficiente y escalable. Su arquitectura modular facilita la integración de nuevos portales de noticias sin necesidad de modificar el flujo general del sistema, garantizando flexibilidad en la captura de datos.

Para estructurar la información obtenida, se definió un **formato de almacenamiento unificado**, asegurando que todas las fuentes sigan el mismo esquema de datos. Además, se implementó un **pipeline de procesamiento común**, lo que permite que cada nuevo portal de noticias solo requiera una configuración específica para su extracción, sin afectar el procesamiento global.

En la obtención de datos, se empleó **XPath** (*XML Path Language*), un lenguaje que permite localizar y extraer elementos dentro del contenido HTML de una página. Esto facilita la identificación de información clave, como títulos, fechas y descripciones de noticias, de forma precisa y estructurada. XPath fue elegido sobre otras técnicas como *CSS Selectors* debido a su capacidad de manejar estructuras complejas de manera más eficiente.

Gracias a esta estrategia, la adición de nuevas fuentes al sistema es un proceso ágil, ya que únicamente es necesario definir la forma de acceso a los datos sin alterar la lógica del procesamiento. Esto garantiza un scraping flexible, escalable y de fácil mantenimiento, adaptándose a cambios en la estructura de los portales de noticias sin comprometer el flujo de datos del sistema.

2.4 Procesamiento Inteligente

En esta sección se describe el enfoque utilizado para procesar automáticamente las noticias recolectadas, identificando y clasificando aquellas que resultan relevantes para el usuario final. Se presenta la metodología aplicada para la clasificación mediante técnicas de aprendizaje automático, así como los desafíos asociados al procesamiento de texto en lenguaje natural.

2.4.1 Clasificación de Noticias

Para abordar la problemática de **automatizar el reconocimiento y clasificación de noticias de interés**, enfocadas en las categorías *robo*, *incendio*, *accidente vial* y *asesinato*, se aplicaron conocimientos adquiridos en la materia optativa *Taller de Inteligencia Artificial*. En particular, se utilizó la subárea de **Machine Learning** ([Zhou, 2021](#)), optando por la técnica de **Aprendizaje Supervisado** para lograr un mejor control sobre el desempeño del modelo y la precisión en la clasificación.

2.4.1.1 Aprendizaje Supervisado vs. No Supervisado

El **Aprendizaje Supervisado** consiste en entrenar un modelo utilizando un conjunto de datos previamente etiquetado (*dataset*), donde cada ejemplo de entrada tiene una categoría asignada. Esto permite que el modelo aprenda a asociar patrones específicos con cada categoría y luego pueda clasificar nuevos datos con base en ese aprendizaje.

En contraste, el **Aprendizaje No Supervisado** ([Bishop, 2006](#)) trabaja con datos sin etiquetas, agrupando los ejemplos en categorías basadas en similitudes detectadas automáticamente. Si bien este enfoque puede ser útil para encontrar patrones desconocidos, no ofrece el mismo grado de control sobre la clasificación de datos específicos como en este caso, donde se requiere identificar categorías bien definidas.

La elección del **Aprendizaje Supervisado** se debió a que proporciona una mayor precisión y confiabilidad en la asignación de categorías, ya que el modelo aprende directamente a partir de ejemplos concretos de noticias etiquetadas.

2.4.1.2 Construcción del Dataset

Uno de los principales desafíos de utilizar Aprendizaje Supervisado es la necesidad de contar con un **dataset de entrenamiento** con ejemplos representativos del dominio de interés. Durante la investigación, se encontraron bases de datos de noticias en inglés, como el dataset de la **BBC de Inglaterra**, pero no contenían las categorías específicas requeridas para este proyecto.

Debido a la falta de un dataset adecuado en español, se optó por la **creación manual de un dataset** recopilando y clasificando noticias de forma individual. Se estableció un conjunto inicial de aproximadamente **100 noticias por categoría**, asegurando un balance en la distribución de clases para mejorar la capacidad de generalización del modelo.

2.4.1.3 Procesamiento del Texto y NLP

Dado que las noticias están escritas en lenguaje natural, fue necesario aplicar técnicas de **Procesamiento del Lenguaje Natural (NLP - Natural Language Processing)** para transformar los textos en un formato adecuado para el modelo de Machine Learning.

El texto en bruto no puede ser interpretado directamente por un algoritmo de clasificación, por lo que se aplicó un **preprocesamiento** que incluyó:

- **Tokenización:** Separación del texto en palabras individuales.
- **Eliminación de palabras vacías (stopwords):** Eliminación de términos irrelevantes como "el", "de", "y", etc.
- **Lematización:** Reducción de las palabras a su forma base (por ejemplo, "robando" → "robar").
- **Vectorización:** Conversión del texto en representaciones numéricas mediante **TF-IDF (Term Frequency - Inverse Document Frequency)** ([Salton & Buckley, 1988](#)), técnica que asigna pesos a las palabras en función de su importancia dentro del conjunto de datos.

2.4.1.4 Entrenamiento del Modelo Clasificador

Con el dataset preparado y preprocesado, se procedió a entrenar diferentes modelos de clasificación para evaluar su desempeño. Se probaron las siguientes técnicas:

- **Multinomial Naïve Bayes (MultinomialNB)** ([Kibriya et al., 2004](#)): Algoritmo probabilístico basado en el teorema de Bayes, eficiente para clasificación de textos pero sensible a desequilibrios en los datos.
- **Regresión Logística (Logistic Regression)** ([Hosmer et al., 2013](#)): Modelo lineal que estima la probabilidad de pertenencia a una categoría específica.
- **Máquinas de Soporte Vectorial (SVM - Support Vector Machine)** ([Cortes & Vapnik, 1995](#)): Algoritmo que encuentra el hiperplano óptimo para separar las clases, especialmente útil cuando los datos no son completamente lineales.

Tras evaluar el desempeño de cada modelo en base a métricas como **precisión, recall y F1-score**, se determinó que el mejor rendimiento se obtuvo con **SVM**, en combinación con la representación numérica de los textos mediante **TF-IDF**.

Este modelo final permite clasificar nuevas noticias de manera precisa, utilizando el mismo preprocesamiento de texto para garantizar consistencia en los datos ingresados al sistema.

2.4.2 Named Entity Recognition (NER)

La correcta extracción de entidades geográficas en el texto fue una de las etapas más complejas del proyecto, debido a la dificultad de identificar ubicaciones precisas relacionadas con los hechos reportados en las noticias. Para abordar esta problemática, inicialmente se evaluó el uso de **SpaCy** ([Explosion, 2025](#)), una biblioteca avanzada de procesamiento de lenguaje natural (*NLP - Natural Language Processing*), ampliamente utilizada en el reconocimiento de entidades nombradas (*NER - Named Entity Recognition*).

2.4.2.1 Evaluación del Uso de SpaCy

SpaCy proporciona modelos preentrenados para el reconocimiento de entidades en español, los cuales pueden identificar nombres de personas (*PER*), organizaciones (*ORG*), ubicaciones (*LOC*), fechas y otros elementos en un texto. Sin embargo, al aplicar estos modelos a las noticias extraídas, se detectaron limitaciones:

1. **Clasificación incorrecta de nombres de calles**

- En la ciudad de **Tandil**, algunas calles llevan nombres de personas o próceres (*Ej.: San Martín, Roca, Mitre*), lo que ocasionaba que el modelo de SpaCy las etiquetara como **PER (Persona)** en lugar de **LOC (Localidad o Ubicación)**.

2. **Falta de reconocimiento de calles específicas**

- En varios casos, nombres de calles de Tandil no eran reconocidos por el modelo, ni como **PER** ni como **LOC**, afectando la identificación precisa de ubicaciones.

Dado que los modelos preentrenados no ofrecían la precisión deseada, se optó por desarrollar una solución personalizada basada en reglas, aprovechando las características de redacción utilizadas en los **portales de noticias oficiales**.

2.4.2.2 Método Basado en Reglas para Extracción de Ubicaciones

Para mejorar la detección de ubicaciones en los textos, se diseñó una estrategia basada en las siguientes observaciones:

1. **Uso de mayúsculas en los nombres de calles**

- En los portales de noticias, los nombres de calles suelen respetar la escritura formal y comienzan en **mayúscula**.
- Se estableció una regla para extraer secuencias de palabras en mayúscula hasta encontrar una palabra en minúscula que no fuera un conector (*Ej.: "de", "y", "del"*).
- Esto permite identificar correctamente nombres como **"López de Osornio"**, evitando segmentaciones erróneas como "López".

2. **Identificación de calles con numeración**

- También se incluyeron expresiones que comienzan con un **número seguido de palabras en mayúscula**, permitiendo detectar calles como **"9 de Julio"**.

3. Filtrado contextual para evitar ubicaciones irrelevantes

- En una noticia pueden mencionarse múltiples ubicaciones, pero no todas están relacionadas con el **lugar del hecho**.
- Para mejorar la precisión, se aplicó un **filtrado basado en contexto**, seleccionando oraciones que contuvieran expresiones que aumentarían la probabilidad de que la ubicación mencionada correspondiera al evento reportado.

2.4.2.3 Estrategia de Geolocalización y Normalización de Direcciones

Una vez extraídas las posibles ubicaciones desde el texto, se implementaron dos estrategias de reconocimiento:

1. **Intersección de calles** (Ej.: "Avenida Marconi y Avellaneda")
2. **Dirección con altura** (Ej.: "Mitre 245")

Con estas direcciones, se realizaron consultas a la **API de Google Maps** ([Google Developers, 2025](#)), utilizando su servicio de *geocoding* para obtener las coordenadas geográficas (*latitud y longitud*). Sin embargo, surgió un nuevo problema:

- La **API de Google Maps** solo reconoce direcciones con el formato completo bien escrito.
- Si una calle estaba escrita de manera incompleta o con variantes informales (Ej.: "Colectora Pugliese" en lugar de "Colectora Sur J. C. Pugliese"), la consulta fallaba.

Para solucionar esto, se implementó un **método de Approximate String Matching (ASM)** ([Yang, Yu, & Kitsuregawa, 2010](#)), basado en la comparación de similitud de cadenas de texto. Se construyó una base de datos con los nombres oficiales de las calles de Tandil y se aplicó ASM para corregir automáticamente las referencias incorrectas.

Ejemplos de corrección:

- "Marconi" → "Avenida Marconi"
- "Colectora Pugliese" → "Colectora Sur J. C. Pugliese"

2.4.2.4 Integración con la Base de Datos

Una vez obtenidas las coordenadas geográficas, se validaron las ubicaciones exitosas y se almacenaron en la base de datos junto con la información de la noticia, incluyendo:

- **Título**
- **Texto completo**
- **Fecha de publicación**
- **URL de la fuente**
- **Ubicaciones geolocalizadas**

Con esto, el sistema garantiza que cada noticia almacenada posea una ubicación precisa, permitiendo que el usuario consulte eventos según su proximidad e intereses.

2.5 Repositorio de Noticias

Para el almacenamiento de la información extraída y procesada en las etapas previas, se optó por utilizar **PostgreSQL** ([PostgreSQL Global Development Group, 2024](#)) como motor de base de datos. Esta decisión se basó en varias razones:

- **Código abierto y comunidad activa:** PostgreSQL es un sistema de gestión de bases de datos relacional (*RDBMS - Relational Database Management System*) de código abierto, ampliamente utilizado en entornos de producción debido a su estabilidad y soporte por parte de una gran comunidad de desarrolladores.
- **Eficiencia y escalabilidad:** Su capacidad para manejar grandes volúmenes de datos y su eficiencia en consultas complejas lo hacen una opción adecuada para el almacenamiento y gestión de noticias con datos asociados, como ubicaciones y tipos de eventos.
- **Soporte para JSON y extensiones avanzadas:** Permite almacenar estructuras flexibles de datos, facilitando futuras expansiones del sistema si fuese necesario.
- **Integración con Django:** PostgreSQL es totalmente compatible con Django, lo que permite una gestión eficiente de los modelos definidos en el ORM (*Object-Relational Mapping*).

La elección de este motor de base de datos también estuvo respaldada por los conocimientos adquiridos en la materia *Base de Datos I*, donde se trabajó con bases de datos relacionales y se comprendieron principios fundamentales de normalización, optimización de consultas y diseño eficiente de esquemas.

2.5.1 Diseño de la Base de Datos

Tomando en cuenta los conceptos estudiados en la materia *Estructura de Almacenamiento de Datos*, se diseñó un esquema que minimiza la redundancia de información mediante la aplicación de técnicas de **normalización**. El modelo de datos está estructurado en entidades bien definidas, con relaciones que permiten mantener la coherencia y eficiencia en la manipulación de los datos.

A continuación, se presenta el esquema relacional de la base de datos (**Figura 2**), donde se observa cómo se organizan las entidades y sus relaciones:

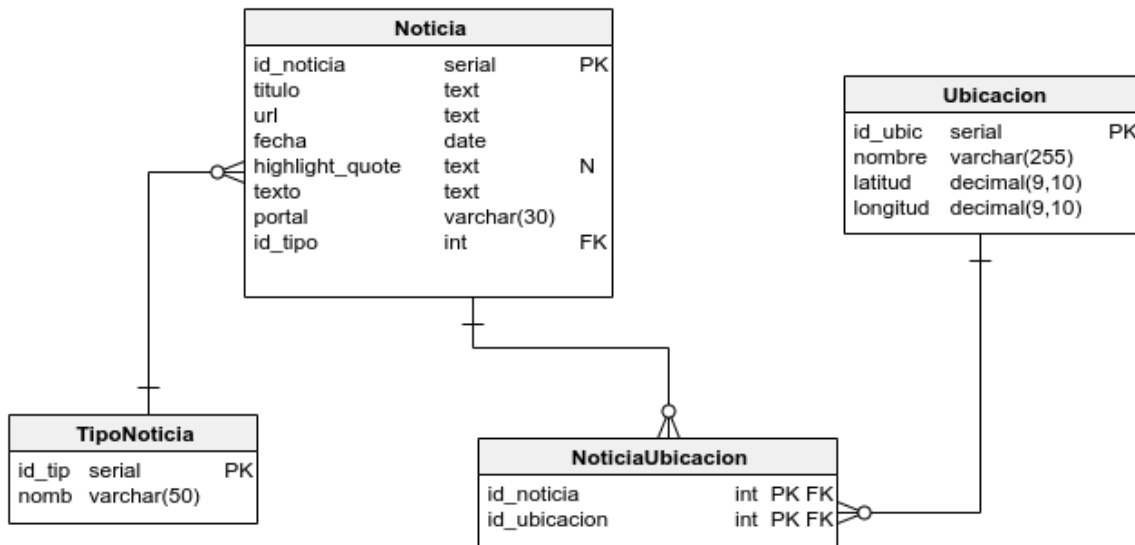


Figura 2. Esquema de la base de datos

El diseño de la base de datos incluye las siguientes entidades principales:

1. Tabla TipoNoticia

Esta tabla almacena las categorías de noticias, asegurando que cada noticia esté correctamente clasificada.

- **id_tipo** (Clave primaria): Identificador único de cada tipo de noticia.
- **nombre**: Nombre de la categoría (por ejemplo, "Accidente Vial", "Robo", "Incendio", "Asesinato").

Esta estructura evita almacenar repetidamente el nombre del tipo de noticia en cada registro, asegurando mayor eficiencia y consistencia en los datos.

2. Tabla Noticia

Representa cada noticia almacenada en el sistema, manteniendo información esencial sobre su origen, contenido y clasificación.

- **id_noticia** (Clave primaria): Identificador único de la noticia.
- **titulo**: Título de la noticia.
- **url**: Enlace a la fuente original de la noticia (único por restricción de integridad).
- **fecha**: Fecha de publicación de la noticia.
- **highlight_quote**: Frase destacada de la noticia (opcional).
- **texto**: Contenido completo de la noticia.
- **portal**: Nombre del medio de comunicación que publicó la noticia.

- **id_tipo** (Clave foránea): Relación con la tabla **TipoNoticia**, indicando la categoría de la noticia.

Al separar las categorías en una tabla independiente y referenciarlas mediante una **clave foránea**, se evita la redundancia y se facilita la gestión de los datos.

3. Tabla Ubicacion

Guarda las coordenadas geográficas de las ubicaciones extraídas mediante el proceso de **Named Entity Recognition (NER)** y **geocoding**.

- **id_ubicacion** (Clave primaria): Identificador único de la ubicación.
- **nombre**: Nombre de la ubicación detectada (ejemplo: "Avenida Marconi y Avellaneda").
- **latitud**: Coordenada de latitud correspondiente.
- **longitud**: Coordenada de longitud correspondiente.

Esta tabla permite almacenar las ubicaciones con sus coordenadas exactas, asegurando que las consultas geoespaciales sean eficientes.

4. Tabla Intermedia NoticiaUbicacion

Dado que una noticia puede referirse a múltiples ubicaciones y una ubicación puede aparecer en varias noticias, se utilizó una **relación muchos a muchos** mediante una tabla intermedia.

- **id_noticia** (Clave foránea): Referencia a la noticia.
- **id_ubicacion** (Clave foránea): Referencia a la ubicación.

Se definió una **restricción de unicidad** para garantizar que no se repitan asociaciones entre una misma noticia y ubicación.

2.5.2 Aplicación de la Normalización

Para garantizar un almacenamiento eficiente, se aplicaron principios de **normalización**, minimizando la redundancia y asegurando la integridad de los datos.

- **Primera Forma Normal (1NF):**
 - Cada campo almacena valores atómicos (sin listas o conjuntos).
 - Las noticias y ubicaciones están separadas en entidades independientes.
- **Segunda Forma Normal (2NF):**
 - Todas las columnas dependen completamente de la clave primaria en cada tabla.
 - Se evita almacenar información repetida sobre categorías de noticias o ubicaciones.
- **Tercera Forma Normal (3NF):**
 - Se eliminaron dependencias transitivas, asegurando que cada tabla contenga únicamente atributos directamente relacionados con su clave primaria.

Gracias a esta estructura, se optimiza el rendimiento de las consultas, se facilita la escalabilidad del sistema y se garantiza que los datos almacenados sean coherentes y fáciles de gestionar.

2.6 Exposición y Visualización de Datos

Para que la aplicación pueda manejar noticias de forma eficiente y flexible, se desarrolló un **backend** utilizando **Django**. Este backend actúa como el intermediario entre la base de datos y la aplicación web, permitiendo a los usuarios acceder a las noticias y aplicar distintos filtros para encontrar la información que les interese.

2.6.1 Funcionamiento del Backend

El backend proporciona una **API** (Interfaz de Programación de Aplicaciones) que permite a la aplicación web obtener las noticias almacenadas. Esta API responde a solicitudes específicas, devolviendo solo la información que el usuario necesita en cada momento.

2.6.2 Mecanismos de Filtrado de Noticias

Para mejorar la experiencia del usuario, se diseñaron varias formas de filtrar las noticias según diferentes criterios:

1. **Búsqueda por tipo de noticia**
 - Permite seleccionar noticias de una categoría específica, como "Accidente de tráfico", "Incendio" o "Robo".
2. **Filtrado por fecha de publicación**
 - Se pueden buscar noticias publicadas en un rango de fechas determinado.
 - Ejemplo: Si alguien quiere ver sólo las noticias publicadas entre el 1 y el 10 de marzo de 2025, puede definir ese período y obtener solo esas noticias.
3. **Noticias cercanas a la ubicación del usuario**
 - Si el usuario quiere conocer noticias que ocurren cerca de su ubicación actual, puede activar este filtro.
4. **Combinación de filtros**
 - Es posible usar varios filtros al mismo tiempo.
 - Ejemplo: Buscar noticias de "Robo" publicadas en la última semana y que estén cerca de la ubicación del usuario.

2.6.3 Comunicación entre la Aplicación Web y el Backend

La aplicación web (desarrollada con **Next.js**) se comunica con el backend mediante la API. Cada vez que el usuario selecciona un filtro, la aplicación envía una solicitud al backend, que procesa la información y devuelve solo las noticias que cumplen con los criterios elegidos.

Por ejemplo, si un usuario elige ver noticias de "Incendio" publicadas en la última semana, la aplicación envía esa solicitud al backend, y este devuelve solo esas noticias. Esto permite que la aplicación sea rápida y eficiente, mostrando solo lo necesario sin recargar información innecesaria.

2.6.4 Interfaz Web y Experiencia de Usuario

La aplicación web fue diseñada para proporcionar a los usuarios una experiencia intuitiva y eficiente al interactuar con las noticias. Se utilizó **Next.js** como framework de front-end debido a su rendimiento optimizado, facilidad de desarrollo, garantizando una navegación fluida y rápida.

Al ingresar a la aplicación, los usuarios son recibidos con una interfaz que combina un **mapa interactivo** y un **menú lateral desplegable** (sidebar). Este diseño permite aplicar filtros de manera sencilla sin necesidad de recargar la página, brindando una experiencia dinámica. El mapa, basado en **OpenStreetMap** ([OpenStreetMap, 2025](#)), muestra las ubicaciones de las noticias mediante marcadores visuales en forma de globos rojos, similares a los de Google Maps.

El sistema de filtrado permite a los usuarios seleccionar noticias según diferentes criterios, como categoría, rango de fechas o cercanía a una ubicación específica. Al aplicar los filtros, los marcadores en el mapa se actualizan en tiempo real, reflejando únicamente las noticias que cumplen con las preferencias seleccionadas.

Cuando un usuario hace clic en un marcador, se despliega una lista con los títulos de todas las noticias relacionadas con esa ubicación. En casos donde múltiples portales informativos hayan reportado la misma noticia, estos serán agrupados dentro del mismo marcador, permitiendo al usuario visualizar todas las fuentes disponibles. Al seleccionar un título, el usuario será redirigido al portal original donde se publicó la noticia.

Este enfoque garantiza que los usuarios no solo puedan encontrar noticias de manera eficiente, sino que también tengan acceso a diversas fuentes para contrastar información y ampliar su perspectiva sobre los hechos reportados. La integración entre el backend desarrollado en **Django** y el front-end en **Next.js** permite una comunicación fluida mediante API, asegurando que la información se actualice de forma rápida y precisa.

En conjunto, la aplicación web no solo proporciona un acceso estructurado a la información geolocalizada, sino que también mejora la interacción con las noticias, ofreciendo una forma innovadora de explorarlas en función del contexto geográfico y temporal del usuario.

A continuación, se presentan capturas de pantalla de la aplicación que ilustran tanto la interfaz inicial como la visualización de los eventos tras aplicar los filtros correspondientes.

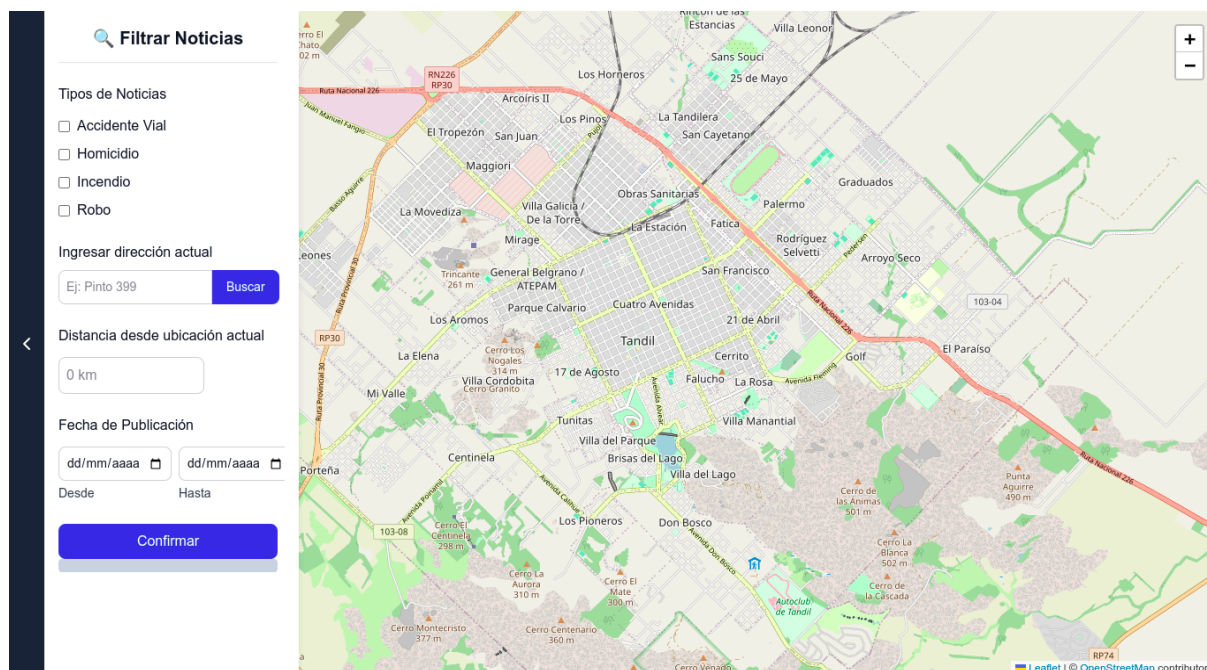


Figura 3. Interfaz principal del sistema con panel de filtros y mapa de eventos

En la **Figura 3** se observa la vista inicial de la aplicación, donde se presenta el mapa interactivo junto al panel lateral de filtros. Este diseño permite seleccionar categorías, definir rangos de fechas o establecer una ubicación específica de interés.

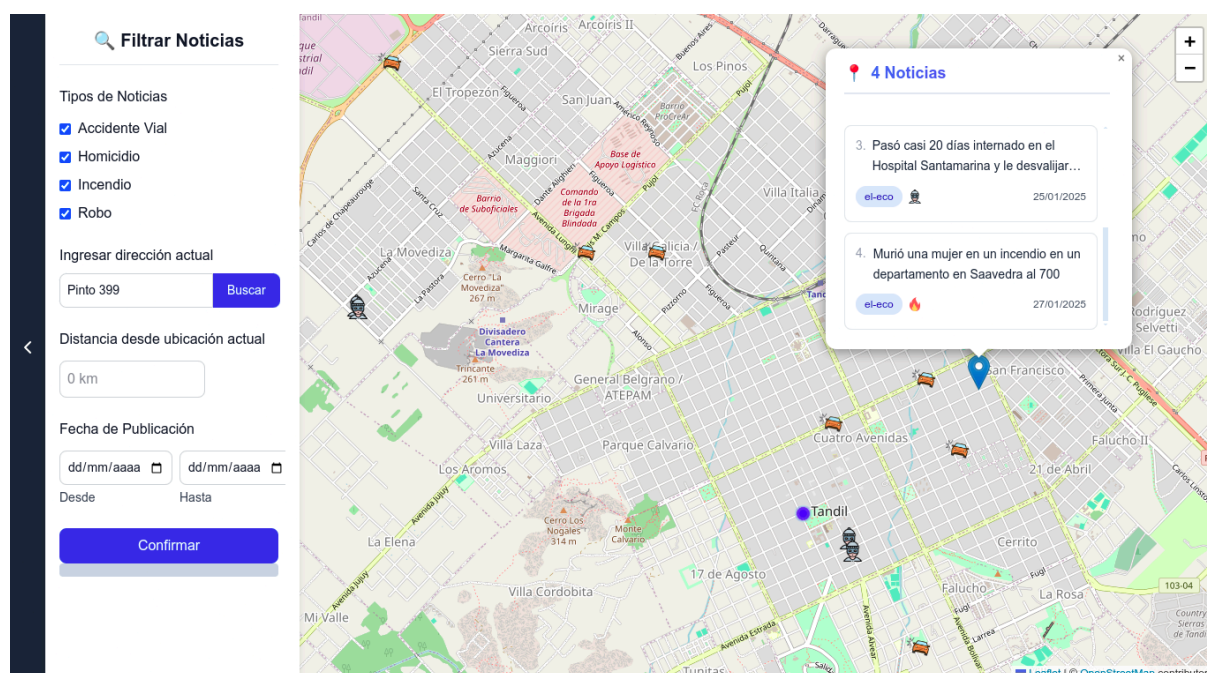


Figura 4. Visualización de eventos geocalizados tras aplicar filtros

En la **Figura 4** se observa cómo los eventos se representan en el mapa mediante íconos específicos según el tipo de evento. Cuando en una misma ubicación se concentran varios eventos de distinta naturaleza, estos se agrupan y se muestran mediante un globo azul. Al hacer clic sobre cualquiera de

estos marcadores, se despliega una lista con las noticias correspondientes a esa ubicación, lo que permite al usuario explorar y comparar información proveniente de distintas fuentes.

3. Implementación

La presente sección describe detalladamente cómo se llevó a cabo la implementación técnica de los distintos módulos del sistema desarrollado. A partir de la arquitectura modular previamente definida, se implementaron herramientas específicas para cada etapa del proceso: desde la recolección de datos hasta su visualización en la aplicación web.

A continuación, se detalla la implementación práctica de cada componente, con foco en las decisiones tecnológicas, fragmentos representativos de código y estrategias adoptadas para garantizar la eficiencia, escalabilidad y mantenibilidad del sistema.

3.1 Estructura General del Sistema

El sistema fue desarrollado siguiendo una arquitectura modular, organizada en distintas carpetas según la responsabilidad de cada componente. Esta estructura favorece la claridad, el mantenimiento y la escalabilidad del proyecto, permitiendo que cada módulo evolucione de forma independiente sin interferir en el resto.

La organización general del proyecto se presenta a continuación:

```
./
├── backend/
├── data/
├── docker/
│   ├── backend/
│   ├── frontend/
│   └── cron/
├── frontend/
├── .env
└── docker-compose.yml
```

Dentro del directorio **backend/** se agrupan tanto el backend web como los módulos de procesamiento y clasificación. Por un lado, se encuentra el proyecto Django (**app/**), encargado de gestionar el acceso a la base de datos y ofrecer una API REST consumida por el frontend. Por otro, se ubican los módulos **scraper/**, **classifier/** y **ner/**, que implementan la lógica del scraping de noticias, la clasificación automática y la detección geográfica, respectivamente.

El archivo **main.py**, ubicado también en **backend/**, orquesta el funcionamiento completo de estos tres componentes. Su ejecución está a cargo del contenedor cron, el cual se configura como un servicio independiente dentro de **docker-compose.yml**. Este contenedor lanza de forma periódica el procesamiento de noticias y se comunica con Django para registrar los resultados en la base de datos.

El directorio **data/** contiene archivos auxiliares, como la base de datos persistente de PostgreSQL y el archivo **calles_preprocesado.txt**, necesario para el reconocimiento y normalización de ubicaciones.

También actúa como espacio de intercambio entre módulos, almacenando temporalmente archivos como **news.jsonl**, resultado del scraping, que luego será consumido por el clasificador.

Por su parte, la carpeta **frontend/** aloja la aplicación desarrollada en Next.js. Esta interfaz permite visualizar las noticias geolocalizadas mediante un mapa interactivo, y ofrece funcionalidades como el filtrado por categoría, fecha y cercanía. La comunicación con el backend se realiza mediante peticiones HTTP a la API expuesta por Django.

Toda la solución está contenida dentro de un entorno dockerizado. El archivo **docker-compose.yml** define los servicios principales del sistema: la base de datos (db), el backend (django), el frontend (next.js) y el servicio cron encargado de la automatización del flujo de datos. La configuración específica de cada servicio se encuentra en la carpeta **docker/**, con sus respectivos **Dockerfile**. Además, se utiliza un archivo **.env** para centralizar las variables de entorno sensibles (credenciales, claves de API, parámetros de conexión), lo cual facilita la portabilidad y configuración entre entornos de desarrollo y producción.

Esta segmentación clara entre componentes, sumada al uso de contenedores y buenas prácticas de configuración, permite una implementación robusta, escalable y fácil de desplegar.

3.2 Adquisición de datos

Para automatizar la recolección de noticias desde portales web locales, se implementó un sistema de scraping utilizando el framework **Scrapy**, reconocido por su eficiencia en la extracción estructurada de datos desde páginas HTML. La estrategia adoptada se basó en desarrollar spiders específicos para cada sitio, lo que permitió adaptar la lógica de recolección a las particularidades de cada portal.

En esta primera etapa del proyecto se desarrollaron dos spiders: uno para el portal [ABC Hoy](#) y otro para [El Eco](#), adaptados a la estructura de cada sitio. En ambos casos, los spiders acceden a las secciones principales del sitio y recorren los artículos disponibles, obteniendo de cada uno los atributos necesarios para el procesamiento posterior: **título, URL, fecha, frase destacada (si está presente), cuerpo completo de la noticia y el nombre del portal**.

Para lograr una extracción precisa, se utilizó el lenguaje XPath, que permitió seleccionar de manera robusta los elementos relevantes del HTML. Cada spider cuenta con su lógica completamente separada, adaptada a la estructura interna del portal correspondiente. A modo de ejemplo, en el caso del spider de ABC, se descartaron enlaces a contenidos promocionales (como los acortadores bit.ly) y se implementó una navegación que accede directamente al detalle de cada noticia. Desde allí se extraen los campos con expresiones XPath como las siguientes:

```
new_item['date'] = response.xpath("//li[@class =
'entry__meta-date']/text()").get()
new_item['highlight_quote'] = response.xpath("//div[@class =
'row']/div/div/h5/text()").get() or ""
new_item['text'] =
"".join(response.xpath("//div[@class='entry__article']//text()").ge
tall()) or ""
```

Para mantener la limpieza y coherencia de los datos, los items recolectados son procesados a través de un pipeline de Scrapy (**NewsscraPerPipeline**) que remueve caracteres no imprimibles (como), recorta espacios sobrantes y elimina elementos irrelevantes del texto, como etiquetas internas del sitio. En el caso particular del portal El Eco, se aplica además una normalización de fechas que convierte formatos como "14 de marzo de 2025" en el estándar "14/03/2025", lo cual es fundamental para su posterior interpretación por el backend.

La estructura del Item utilizado, llamado **NewItem**, fue definida con campos explícitos para cada atributo esperado. Esta estructura actúa como intermediario entre el spider y los módulos de procesamiento que vendrán a continuación, permitiendo mantener los datos organizados y fácilmente accesibles.

Este item fue definido en el archivo items.py dentro del proyecto Scrapy, donde se declararon los campos url, title, date, highlight_quote, text y portal utilizando la clase scrapy.Field(). De esta manera, se establece un esquema claro y consistente para todos los datos recolectados, lo cual facilita su procesamiento posterior en los módulos de clasificación y geolocalización.

Uno de los principales desafíos surgió con el portal **ABC Hoy**, que implementa mecanismos de protección contra scraping, incluyendo bloqueo de IPs y detección de patrones de acceso sospechosos. Inicialmente se consideró introducir retardos entre solicitudes para evitar bloqueos, pero esta solución comprometía el rendimiento del sistema. En respuesta, se optó por la integración de un **middleware personalizado** que rota agentes de usuario utilizando la API de *ScrapeOps*. Este componente genera encabezados HTTP falsos con agentes de usuario aleatorios, simulando el comportamiento de distintos navegadores y dispositivos. Su implementación, incluida en el archivo middlewares.py, permitió mejorar significativamente la estabilidad del scraping en este portal, sin penalizar el tiempo de ejecución.

La ejecución del scraping se realiza desde el archivo main.py, utilizado por el contenedor cron, encargado de orquestrar el procesamiento de datos. Desde este archivo se invoca una función (**run_spiders**) que instancia y ejecuta los spiders utilizando la clase **CrawlerProcess**, propia de Scrapy. De esta manera, la recolección se realiza de forma programática y controlada, sin necesidad de invocar el comando scrapy crawl de manera manual. Durante este proceso, las noticias extraídas se guardan en un archivo 'news.jsonl' ubicado en el directorio 'data/', el cual luego será consumido por el clasificador. Un fragmento representativo del contenido de este archivo se muestra a continuación:

```
{
    "url":
    "https://www.abchoy.com.ar/leernota/203176/triple_choque_en_avenid
a_bolivar_y...",
    "title": "Triple choque en Avenida Bolívar y Juncal",
    "date": "2025-04-16",
    "highlight_quote": "Un siniestro vial se produjo en la
intersección de la Avenida Bolívar y ...",
    "text": "Según informó la policía, el siniestro involucró a
tres vehículos: un Peugeot 408...",
    "portal": "abc"
}
```


Cabe destacar que el sistema, en su estado actual, no cuenta con un manejo explícito de errores en el scraping. En caso de que un portal modifique su estructura interna (como sucedió en varias ocasiones con El Eco), el scraper puede fallar silenciosamente o extraer información incompleta. Esta situación se detectó al observar que las noticias obtenidas contenían cuerpos vacíos, lo que motivó la revisión manual de los selectores XPath. Estas experiencias reafirmaron la importancia de monitorear periódicamente el rendimiento del scraping y establecer mecanismos de alerta para futuras versiones.

En síntesis, la implementación del scraper permitió establecer una fuente automatizada y estructurada de datos para alimentar el sistema, contemplando los desafíos propios de interactuar con portales web dinámicos, con estructuras dispares y en constante evolución.

Con las noticias extraídas y almacenadas en el archivo 'news.jsonl', el siguiente paso consiste en procesarlas automáticamente para identificar su categoría, mediante técnicas de aprendizaje supervisado. Esta tarea es abordada en la etapa de clasificación de noticias, desarrollada a partir de modelos entrenados previamente.

3.3 Clasificación Automática del Tipo de Noticia

Una vez recolectadas las noticias mediante el sistema de scraping, el siguiente paso en la cadena de procesamiento consiste en **asignar automáticamente una categoría** semántica a cada una de ellas. Esta clasificación permite estructurar la información y habilita funcionalidades clave del sistema, como el filtrado por tipo de incidente en la interfaz web. Para lograr este objetivo, se aplicaron técnicas de **aprendizaje supervisado**, entrenando modelos de clasificación sobre un conjunto de noticias previamente etiquetadas.

El dataset de entrenamiento fue construido de forma manual, seleccionando y clasificando artículos provenientes de diferentes portales de noticias además de los implementados que se utilizaron en la etapa de scraping. Este conjunto está compuesto por un total de **554 noticias**, cada una asociada a una de las cinco categorías principales consideradas por el sistema: robo, accidente vial, asesinato, incendio y otro. La distribución de clases no es perfectamente balanceada, pero se procuró mantener una representación suficiente de cada una para permitir un entrenamiento robusto del modelo. La siguiente tabla resume la cantidad de ejemplos por categoría:

Distribución de clases en el dataset	
otro	147
robo	108
accidente vial	102
asesinato	100
incendio	97

Tabla 1. Distribución de clases en el dataset

Cada entrada del dataset incluye el título, la cita destacada, el cuerpo de la noticia y metadatos adicionales como la fecha, el portal de origen y la URL. Durante el preprocesamiento, estos campos fueron unificados en un único texto concatenado (`full_text`) que representa el contenido completo que será analizado por el modelo. A partir de este conjunto, se evaluaron distintas combinaciones de técnicas de vectorización y algoritmos de clasificación, con el objetivo de encontrar la configuración con mejor desempeño.

3.3.1 Evaluación de Técnicas y Selección del Modelo

A partir del dataset etiquetado, se procedió a comparar distintas combinaciones de técnicas de vectorización y algoritmos de clasificación. El objetivo de este análisis fue determinar cuál ofrecía el mejor rendimiento para predecir correctamente la categoría de una noticia basada en su contenido textual.

Para ello, se evaluaron dos enfoques de representación del texto: **Bag of Words (BoW)** ([Wallach, 2006](#)) y **TF-IDF**, ambos ampliamente utilizados en tareas de procesamiento de lenguaje natural. En paralelo, se probaron tres modelos de clasificación: **Support Vector Machines (SVM)** con kernel lineal, **Naive Bayes** y **Regresión Logística**. Todas las combinaciones posibles fueron entrenadas y evaluadas utilizando una división estratificada de los datos (70% entrenamiento, 30% prueba), preservando la proporción de clases.

Previo a la vectorización, el texto fue normalizado mediante el modelo de procesamiento lingüístico **spaCy**, aplicando minúsculas, lematización, eliminación de stopwords y filtrado de tokens no alfabéticos. Este preprocesamiento permitió mejorar la calidad de los vectores generados por BoW y TF-IDF.

Un fragmento del script que implementa este análisis se presenta a continuación:

```
def preprocess_text(text):
    doc = nlp(text.lower())
    tokens = [
        token.lemma_ for token in doc
        if not token.is_stop and token.is_alpha
    ]
    return " ".join(tokens)

df["full_text"] = df["title"] + " " + df["highlight_quote"] + " "
+ df["text"]
df["full_text"] = df["full_text"].apply(preprocess_text)

# Vectorizadores
vectorizer_bow = CountVectorizer()
X_bow = vectorizer_bow.fit_transform(df["full_text"])

vectorizer_tfidf = TfidfVectorizer()
X_tfidf = vectorizer_tfidf.fit_transform(df["full_text"])
```

Cada vectorización fue utilizada para entrenar los modelos, evaluando su desempeño con el conjunto de prueba. La métrica principal de comparación fue la **f1-score**, calculada por clase, junto con la precisión y el recall promedio.

3.3.2 Comparación de Modelos y Vectorizadores

Una vez preparado y preprocesado el dataset, se procedió a evaluar distintas combinaciones de técnicas de representación del texto (**BoW** y **TF-IDF**) junto con tres algoritmos de clasificación: **Support Vector Machines (SVM)**, **Naive Bayes (NB)** y **Logistic Regression (LG)**.

A continuación se presenta una tabla resumen de métricas clave para cada combinación, utilizando un conjunto de prueba estratificado que preserve la proporción de clases:

Modelo	Vectorizador	Accuracy	F1-score	Precisión	Recall
SVM	TF-IDF	0.89	0.90	0.90	0.89
SVM	BoW	0.85	0.85	0.86	0.85
NB	TF-IDF	0.86	0.86	0.86	0.87
NB	BoW	0.86	0.86	0.86	0.87
LG	TF-IDF	0.86	0.86	0.87	0.86
LG	BoW	0.85	0.85	0.85	0.86

Tabla 2. Métricas clave por combinación de modelo y vectorizador

- **Accuracy** representa el porcentaje total de predicciones correctas.
- **F1-score** es la media armónica entre precisión y recall, y es especialmente útil cuando las clases están desbalanceadas.
- **Precisión** mide cuántas de las predicciones realizadas para una clase fueron correctas.
- **Recall** indica cuántos de los casos reales de una clase fueron correctamente identificados por el modelo.

Para visualizar mejor el rendimiento, se elaboró un gráfico de barras comparando el **F1-score promedio** por modelo y vectorizador (ver **Figura 5**).

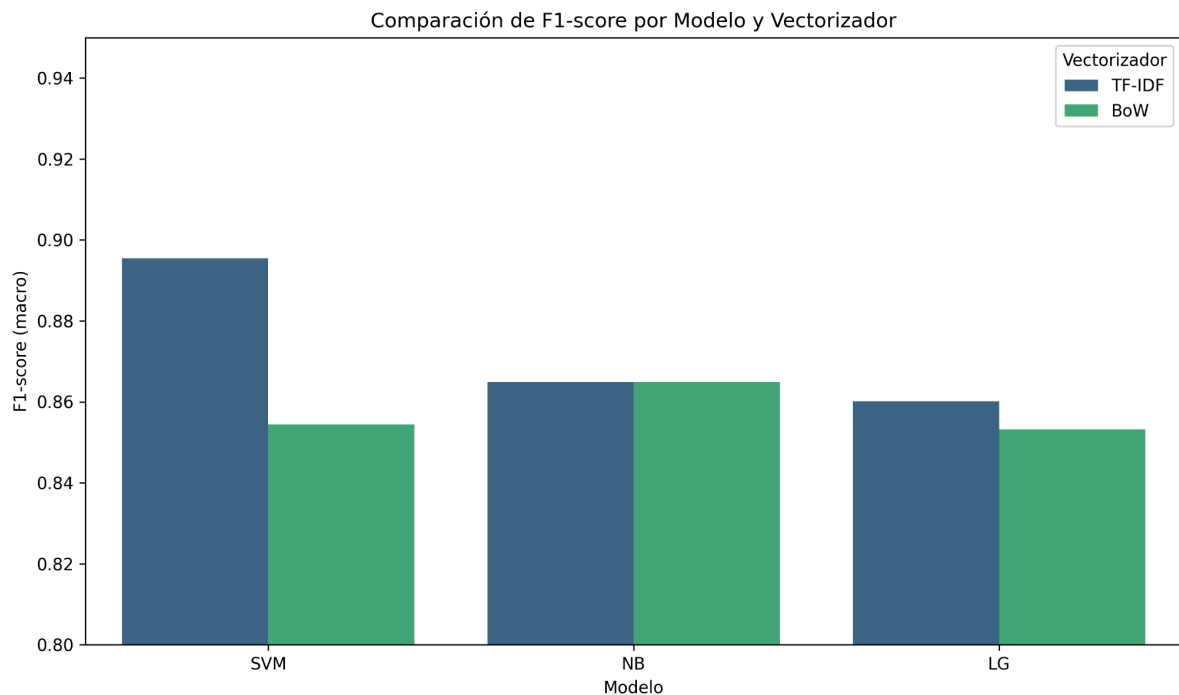


Figura 5. Comparación de F1-score promedio entre modelos y técnicas de vectorización

3.3.3 Desempeño del Modelo Seleccionado (SVM + TF-IDF)

Entre todas las combinaciones evaluadas, la que demostró **el mejor rendimiento global fue SVM con vectorización TF-IDF**, alcanzando un F1-score promedio de **0.90** y una precisión general del **89%**.

En la siguiente tabla se detallan las métricas específicas por clase para este modelo:

Clase	Precisión	Recall	F1-score
Otro	0.84	0.86	0.85
Accidente vial	0.93	0.90	0.92
Robo	0.87	0.84	0.86
Incendio	0.94	1.00	0.97
Asesinato	0.90	0.87	0.88

Tabla 3. Métricas por clase (SVM con TF-IDF)

Se destaca el excelente desempeño en clases como **"incendio"** (F1 = 0.97) y **"accidente vial"** (F1 = 0.92), donde el modelo demostró una capacidad notable para identificar correctamente los patrones

semánticos asociados. Por otro lado, la clase "**otro**" mostró un F1-score ligeramente inferior (0.85), lo cual puede atribuirse a su naturaleza más ambigua y heterogénea, que agrupa casos menos estructurados o residuales.

En contraste, otros modelos como Naive Bayes o Regresión Logística mostraron un desempeño apenas inferior, especialmente en la distinción entre categorías semánticamente cercanas. Por ejemplo, Naive Bayes con TF-IDF alcanzó un accuracy del 86%, pero mostró mayor variación entre clases.

Como resultado de este análisis comparativo, se decidió utilizar **SVM con representación TF-IDF** como modelo principal para el sistema. Este fue entrenado sobre el conjunto completo de datos y luego serializado mediante la librería **joblib**, junto con su vectorizador correspondiente, para ser utilizado en producción.

La serialización del modelo y el vectorizador se realizó utilizando la librería **joblib**, permitiendo su reutilización sin necesidad de reentrenamiento. El siguiente fragmento ilustra cómo se almacenaron ambos componentes luego del entrenamiento:

```
import joblib
joblib.dump(svm_tfidf, "modelo_entrenado/svm_tfidf.pkl")
joblib.dump(vectorizer_tfidf,
"modelo_entrenado/vectorizer_tfidf.pkl")
```

Ambos archivos (**svm_tfidf.pkl** y **vectorizer_tfidf.pkl**) se encuentran almacenados en el directorio **backend/classifier/modelo_entrenado/**.

3.3.4 Análisis Cualitativo del Modelo Seleccionado

Además del análisis cuantitativo de las métricas globales, se realizaron evaluaciones cualitativas para comprender en mayor profundidad el comportamiento del modelo SVM + TF-IDF.

3.3.4.1 Matriz de Confusión Normalizada

Se generó una matriz de confusión normalizada para el conjunto de prueba, que permite observar los errores más comunes de clasificación. Como se muestra en la **Figura 6**, la clase "robo" presentó cierta confusión con "asesinato", lo cual puede explicarse por la similitud semántica entre noticias de contenido violento o policial.

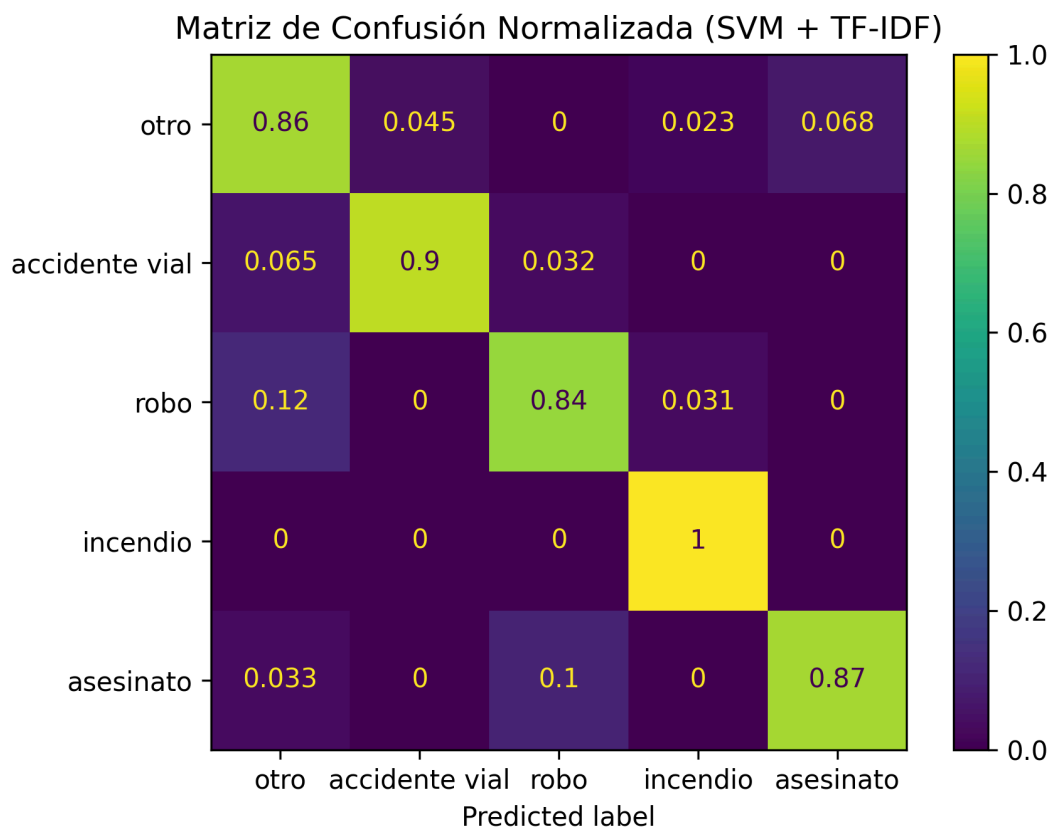


Figura 6. Matriz de confusión normalizada del modelo SVM + TF-IDF.

3.3.4.2 Ejemplos de Errores de Clasificación

Se analizaron manualmente algunas instancias mal clasificadas. En general, se trató de textos ambiguos o poco informativos, donde el modelo carecía de suficientes pistas para una predicción correcta. Un ejemplo típico fue:

"Control vehicular tránsito infracción caso alcoholemia inspector"

Clasificada como: **otro**

Etiqueta real: **accidente vial**

Este tipo de casos sugiere que una mejora en la cobertura léxica o un aumento del volumen de entrenamiento podría reducir errores.

3.3.4.3 Palabras Clave por Clase

Se extrajeron las características (términos) más influyentes para cada clase utilizando los coeficientes del modelo SVM. Esto permitió interpretar el comportamiento del clasificador. Algunas de las palabras más representativas por clase fueron:

- **Accidente vial:** accidente, conductor, choque, vehículo, tránsito.
- **Robo:** estafa, pfa, electrodoméstico, robo.
- **Incendio:** incendio, bomberos, fuego, llamas.

- **Asesinato:** parricidio, arma, francisco, muerto.
- **Otro:** tandil, actividad, evento, arte, ciudad.

Esta información resulta valiosa para comprender cómo el modelo toma decisiones, además de evidenciar la relevancia de ciertos términos en cada categoría.

Es importante destacar que algunas palabras, como “francisco” en la clase “asesinato”, pueden aparecer como características relevantes debido a su frecuencia en noticias específicas dentro del conjunto de entrenamiento. En este caso, el término podría estar relacionado con un caso particular (por ejemplo, una víctima o acusado llamado Francisco) que tuvo alta cobertura y quedó estadísticamente vinculado a esa categoría. Este tipo de asociaciones, aunque lógicamente neutras, reflejan cómo el modelo capta correlaciones textuales sin comprender el significado real, lo que enfatiza la necesidad de análisis crítico y revisión humana del comportamiento del clasificador.

Una posible línea de mejora sería aplicar técnicas de reconocimiento de entidades nombradas (NER) para detectar y filtrar nombres propios o lugares específicos. Esto permitiría reducir el peso de términos que aparecen por correlación contextual en casos puntuales, favoreciendo una generalización más robusta del modelo.

3.3.4 Integración del Modelo en el Sistema

Una vez seleccionado y entrenado el modelo definitivo, se implementaron los módulos necesarios para integrarlo al sistema de procesamiento de noticias. Esta etapa implicó encapsular la lógica de clasificación en componentes reutilizables, manteniendo una separación clara entre el preprocesamiento, el modelo y la lógica de orquestación.

El archivo **text_classifier.py** define una clase **TextClassifier**, que permite cargar el modelo entrenado y su vectorizador desde disco, y realizar predicciones sobre nuevos textos:

```
classifier = TextClassifier()
classifier.load_vectorizer("classifier/modelo_entrenado/vectorizer_tfidf.pkl")
classifier.load_model("svm",
"classifier/modelo_entrenado/svm_tfidf.pkl")
categoria = classifier.predict(texto_preprocesado, "svm")
```

Por su parte, el archivo **preprocesamiento.py** implementa una clase **Preprocessor**, que utiliza SpaCy para normalizar el texto de entrada mediante lematización, conversión a minúsculas y eliminación de stopwords:

```
preprocessor = Preprocessor()
texto_preprocesado = preprocessor.preprocess_text(texto_crudo)
```

La lógica general se encuentra encapsulada en **clasificar_noticias.py**, que recibe un archivo **.jsonl** con las noticias extraídas, aplica el preprocesamiento y utiliza el clasificador para asignar una categoría a cada una. Además, se realiza un filtrado que descarta las noticias clasificadas como "otro", ya que no se consideran relevantes para la visualización en el sistema:

```
from classifier.clasificar_noticias import clasificar_noticias
noticias_filtradas = clasificar_noticias("../data/news.jsonl")
```

Este archivo es utilizado desde el módulo **main.py**, que es ejecutado por el servicio cron. De esta manera, el sistema logra integrar el scraping, la clasificación automática y el almacenamiento en base de datos de manera completamente automatizada.

3.4 Detección y Georreferenciación de Entidades Geográficas

Una de las etapas más complejas e importantes del proyecto fue la extracción automática de entidades geográficas a partir del contenido textual de las noticias. El objetivo de este módulo es identificar lugares mencionados en los artículos —como direcciones e intersecciones— y obtener sus coordenadas mediante una consulta a la API de Google Maps. Esta información es esencial para representar los eventos sobre un mapa en la interfaz final del sistema.

El sistema está compuesto por los siguientes archivos, ubicados en `./backend/ner/`:

```
./backend/ner/
├── location_components.py          # Define clases Direccion e
Interseccion
└── news_location_extractor.py # Lógica principal de extracción
```

Además, se apoya en un archivo externo de referencia de calles: `./data/calles_preprocesado.txt`.

3.4.1 Enfoque Implementado

Dado que las noticias suelen describir incidentes urbanos en forma textual, se diseñó una estrategia basada en reglas y procesamiento lingüístico para extraer entidades del tipo:

- Direcciones (por ejemplo: *Avenida España 1356*)
- Intersecciones (por ejemplo: *Yrigoyen y Mitre*)

Para identificar estas entidades, se implementaron dos componentes principales:

- **location_components.py**, que define las clases **Direccion** e **Interseccion**, ambas heredan de **Ubicacion** y encapsulan tanto su validación como su conversión a texto.
- **news_location_extractor.py**, que contiene toda la lógica de procesamiento textual, extracción de entidades, validación de calles y consulta a la API de Google Maps para obtener las coordenadas geográficas.

3.4.2 Normalización de Nombres de Calles

Una particularidad clave del proceso es que se hizo uso de un archivo adicional `calles_preprocesado.txt`, con todas las calles reales de la ciudad de Tandil. Este recurso fue utilizado para realizar *string matching* y asegurar que las direcciones capturadas puedan ser reconocidas por

la API de geolocalización. Por ejemplo, si una noticia menciona "*colectora pugliese*", esta se transforma automáticamente en "*colectora sur j. c. pugliese*" antes de ser enviada a la API, lo que mejora significativamente la tasa de éxito.

3.4.3 Flujo de Procesamiento

Una vez clasificadas las noticias, se ejecuta el proceso de extracción automática de entidades geográficas. Este proceso tiene como objetivo identificar menciones explícitas de lugares en los textos, como direcciones o intersecciones, y obtener sus coordenadas para su posterior visualización en un mapa.

Este flujo es ejecutado automáticamente desde **main.py**, luego de la clasificación de noticias, mediante la siguiente llamada:

```
from ner.news_location_extractor import extract_news_locations
noticias_con_ubicacion =
extract_news_locations(noticias_filtradas)
```

El procedimiento completo se desarrolla en las siguientes etapas:

1. **Preprocesamiento del texto:** Se separa en oraciones y se estandarizan abreviaturas comunes (como Av. o Pje.).
2. **Filtrado contextual:** Se seleccionan solo aquellas oraciones que contienen palabras clave asociadas a eventos relevantes ("ocurrió", "incendio", "choque", etc.).
3. **Extracción de entidades:** Se detectan direcciones e intersecciones usando patrones sobre mayúsculas, conectores y números.
4. **Validación de calles:** Se utiliza *string matching* para verificar que las calles mencionadas existan en el archivo `calles_preprocesado.txt`.
5. **Geocodificación:** Si las ubicaciones son válidas, se consulta la API de Google Maps para obtener coordenadas.
6. **Filtrado de duplicados:** Se eliminan ubicaciones muy cercanas entre sí para evitar redundancias.

Cada noticia se convierte en un único texto unificado compuesto por el título, la cita destacada y el cuerpo de la noticia. Este texto se divide en oraciones usando el siguiente método:

```
sentences = TextProcessor.preprocess_text(text)
```

Luego, a partir de esas oraciones, se detectan posibles entidades geográficas (direcciones e intersecciones) mediante reglas basadas en el uso de mayúsculas, conectores frecuentes y palabras clave contextuales como "incendio", "choque" o "altura". Cada oración se filtra utilizando la función **is_sentence_valid()**, que evalúa si contiene alguna de las palabras clave definidas en **KEY_EVENTS**:

```
@staticmethod
def is_sentence_valid(text):
    return any(event in text.lower() for event in KEY_EVENTS)
```


Esto permite ignorar menciones de lugares que no estén directamente asociadas a un evento. Solo las oraciones filtradas se analizan para extraer ubicaciones con:

```
event_locations = LocationExtractor.extract_location(sentences)
```

Este procedimiento se encuentra encapsulado en la función **process_news()**:

```
def process_news(news):
    news_with_locations = []
    for item in news:
        text = f"{item['noticia']['title']}. {item['noticia']['highlight_quote']}. {item['noticia']['text']}"
        sentences = TextProcessor.preprocess_text(text)
        event_locations = LocationExtractor.extract_location(sentences)
        if event_locations:
            item_copy = item.copy()
            item_copy["ubicaciones"] = event_locations
            news_with_locations.append(item_copy)
    return news_with_locations
```

Este enfoque permite reducir el **ruido semántico** y mejora la **precisión**, ya que se enfoca solo en oraciones con alta probabilidad de contener referencias geográficas vinculadas al evento.

Las entidades extraídas son instancias de las clases **Direccion** o **Interseccion**, ambas derivadas de una clase abstracta llamada **Ubicacion**. Este diseño permite el uso de polimorfismo: todas las entidades comparten los métodos **get_location()** (que devuelve su representación textual) y **validate_street()** (que verifica si la calle existe). Por ejemplo, el método **get_location()** devuelve el formato que será enviado a la API:

```
def get_location(self):
    return f"{self.calle} {self.numero}" # En Direccion
```

Mientras que **validate_street()** utiliza coincidencia difusa para corregir nombres parcialmente escritos:

```
# De Interseccion
def validate_street(self, lista_calles):
    """Verifica si ambas calles existen en la lista usando String Matching."""
    calle1_valida = self.encontrar_calle_mas_similar(self.calle1, lista_calles)
    calle2_valida = self.encontrar_calle_mas_similar(self.calle2, lista_calles)
    if calle1_valida and calle2_valida:
        self.calle1 = calle1_valida
        self.calle2 = calle2_valida
        return True
    return False
```

Gracias a este enfoque, la validación y geocodificación pueden realizarse de forma homogénea sobre cualquier instancia de **Ubicacion**.

3.4.4 Consulta a la API de Geocodificación

Una vez obtenidas y validadas las ubicaciones, se realiza la consulta a la API de Google Maps mediante la clase **GeoLocator**. Este componente también incluye cacheo de resultados y verificación semántica de la respuesta:

```
def get_coordinates(self, location):
    if not location.validate_street(self.valid_streets):
        return None
    ...
    url =
f"https://maps.googleapis.com/maps/api/geocode/json?address={location_str}&components=locality:Tandil|country:AR&key={self.api_key}"
    ...
    if data.get("status") == "OK" and
location.is_correct_answer_api(...):
        return (lat, lng)
```

3.4.5 Extracción Final

El proceso completo se coordina desde la función **extract_news_locations()**, que integra el procesamiento semántico, la validación, la consulta geográfica y la limpieza de ubicaciones redundantes:

```
def extract_news_locations(news):
    valid_streets = load_valid_streets()
    geolocator = GeoLocator(API_KEY, valid_streets)
    filtered_news = []
    news_ner_processed = process_news(news)
    for news_item in news_ner_processed:
        valid_locations = []
        for location in news_item["ubicaciones"]:
            coords = geolocator.get_coordinates(location)
            if coords:
                valid_locations.append({
                    "ubicacion": location,
                    "coordenadas": coords
                })
        if valid_locations:
            valid_locations =
remove_nearby_locations(valid_locations)

        for loc in valid_locations:
```

```

        loc["nombre"] =
str(loc["ubicacion"].get_location()).title()
        del loc["ubicacion"]

    news_item_copy = news_item.copy()
    news_item_copy["ubicacion_validas"] = valid_locations
    if "ubicaciones" in news_item_copy:
        del news_item_copy["ubicaciones"]
    filtered_news.append(news_item_copy)

return filtered_news

```

3.4.6 Dificultades Encontradas

A pesar de los esfuerzos, esta etapa **no garantiza una cobertura perfecta del 100%**, ya que:

- Algunas noticias no mencionan explícitamente ubicaciones,
- Otras lo hacen de forma ambigua (*“una vivienda en la zona de Villa Italia”*),
- Y la API de Google puede fallar si el formato de dirección no es suficientemente preciso.

Además, fue necesario balancear precisión y flexibilidad: un sistema muy estricto perdía ubicaciones válidas; uno muy permisivo generaba falsos positivos. Por este motivo, se adoptó una lógica híbrida que permite reemplazos, correcciones y verificación semántica, pero con límites razonables.

3.4.7 Resultados Esperados

Como resultado, cada noticia clasificada se enriquece con un listado de posibles ubicaciones, incluyendo su nombre y coordenadas. Este dato es posteriormente utilizado por el backend para cargar los eventos geolocalizados y permitir su consulta desde la interfaz web o vía API REST.

A continuación, se muestra un ejemplo del formato final de una noticia enriquecida con ubicaciones geográficas:

```

{
  "noticia": {
    "title": "Accidente en Colectora",
    ...
  },
  "ubicaciones_validas": [
    {
      "nombre": "Colectora Sur J. C. Pugliese 123",
      "coordenadas": [-37.3204352, -59.1033636]
    },
    {
      "nombre": "25 de Mayo & General Rodríguez",
      "coordenadas": [-37.3312437, -59.1338053]
    }
  ]
}

```

```
}
```

3.5 Automatización con Cron en Contenedor

Para ejecutar automáticamente el proceso de scraping, clasificación y carga de noticias, se configuró un contenedor cron dedicado dentro del entorno Docker. Este módulo tiene como finalidad ejecutar el script `main.py` en intervalos regulares sin intervención manual.

3.5.1 Estructura y Configuración

El contenedor se define en el archivo `docker-compose.yml` con el nombre `cron`, y depende del backend para asegurar que esté operativo antes de ejecutar tareas:

```
cron:
  build:
    context: .
    dockerfile: ./docker/cron/Dockerfile
  container_name: cronjob
  env_file:
    - .env
  depends_on:
    - backend
```

Dentro del directorio `docker/cron`, se encuentra el `Dockerfile` que define el entorno del contenedor. Parte de una imagen ligera de Python 3.10 y se le instala el servicio de cron:

```
FROM python:3.10-slim

RUN apt-get update && apt-get install -y cron

WORKDIR /backend

# En docker/cron/Dockerfile
COPY backend/main.py /backend/
COPY backend/scrapper/ /backend/scrapper/
COPY backend/classifier /backend/classifier
COPY backend/ner /backend/ner

RUN mkdir -p /data
COPY data/calles_preprocesado.txt /data/calles_preprocesado.txt

COPY docker/cron/requirements.txt .

COPY docker/cron/crontab.txt /etc/cron.d/scheduled-tasks

RUN chmod 0644 /etc/cron.d/scheduled-tasks
RUN crontab /etc/cron.d/scheduled-tasks
```

```

RUN pip install -r requirements.txt
RUN python -m spacy download es_core_news_sm

COPY docker/cron/entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh
ENTRYPOINT ["/entrypoint.sh"]

```

3.5.2 Programación de Tareas

En el archivo `crontab.txt` se definen dos ejecuciones automáticas del script principal del sistema (`main.py`). Estas tareas están programadas en formato UTC, por lo que deben interpretarse considerando la zona horaria local:

```

0 11 * * * /bin/bash -c "source /etc/profile.d/env_vars.sh &&
/usr/local/bin/python /backend/main.py" | tee -a /var/log/cron.log
0 15 * * * /bin/bash -c "source /etc/profile.d/env_vars.sh &&
/usr/local/bin/python /backend/main.py" | tee -a /var/log/cron.log

```

Esto significa que el sistema ejecutará automáticamente el proceso completo de recolección, clasificación, geolocalización y almacenamiento de noticias **todos los días a las 08:00 y 12:00 del mediodía (hora local Argentina, UTC-3)**.

Nota: La diferencia horaria se debe a que el cron se ejecuta en UTC por defecto dentro del contenedor Docker.

3.5.3 Manejo de Variables de Entorno

Debido a que el entorno cron no carga automáticamente las variables definidas en `.env`, se implementó un script `entrypoint.sh` que genera un archivo con todas las variables disponibles en `/etc/profile.d/env_vars.sh`:

```

#!/bin/bash

echo "#!/bin/bash" > /etc/profile.d/env_vars.sh

printenv | grep -v "no_proxy" | while IFS='=' read -r key value;
do
    escaped_value=$(printf "%s" "$value" | sed 's/"/\\"/g')
    echo "export $key=\"\$escaped_value\"" >>
/etc/profile.d/env_vars.sh
done

chmod +x /etc/profile.d/env_vars.sh

# Iniciamos cron en foreground
cron -f

```

De este modo, cualquier tarea programada que requiera variables de entorno (como la URL del backend o claves de conexión a la base de datos) puede accederse correctamente al hacer `source`.

3.5.4 Dependencias Utilizadas

El contenedor instala un conjunto mínimo de paquetes necesarios para el procesamiento de noticias:

```
pandas
spacy
scikit-learn
scrapy
requests
```

3.6 Repositorio de Noticias

Una vez finalizadas las etapas de clasificación y extracción de entidades, el sistema procede a almacenar las noticias y sus ubicaciones geográficas en una base de datos PostgreSQL. Esta operación se realiza a través del framework Django, utilizando su ORM (Object-Relational Mapping) para interactuar con la base de datos de forma declarativa, segura y eficiente.

3.6.1 Configuración de Conexión en settings.py

Django se conecta a la base de datos PostgreSQL mediante la siguiente configuración ubicada en **settings.py**. Se utiliza el paquete **python-decouple** para mantener las credenciales y parámetros sensibles en variables de entorno:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': config('POSTGRES_DB'),
        'USER': config('POSTGRES_USER'),
        'PASSWORD': config('POSTGRES_PASSWORD'),
        'HOST': config('POSTGRES_HOST', default='localhost'),
        'PORT': config('POSTGRES_PORT', default='5432'),
    }
}
```

Esta práctica mejora la seguridad y portabilidad entre entornos (desarrollo, producción, testing).

3.6.2 Estructura de los Modelos en Django

La estructura del almacenamiento está compuesta por cuatro modelos principales definidos en el archivo **models.py**:

- **TipoNoticia:** Categorías como "Accidente Vial", "Robo", etc.
- **Noticia:** Representa una noticia con su contenido, fecha, URL, etc.
- **Ubicacion:** Contiene el nombre y las coordenadas de una ubicación geográfica.

- **NoticiaUbicacion:** Modelo intermedio para la relación muchos-a-muchos entre noticias y ubicaciones.

Ejemplo resumido del código:

```
class TipoNoticia(models.Model):
    id_tipo = models.AutoField(primary_key=True)
    nombre = models.CharField(max_length=50, unique=True)

class Noticia(models.Model):
    id_noticia = models.AutoField(primary_key=True)
    titulo = models.TextField()
    url = models.TextField(unique=True)
    fecha = models.DateField()
    highlight_quote = models.TextField(blank=True, null=True)
    texto = models.TextField()
    portal = models.CharField(max_length=30)
    id_tipo = models.ForeignKey(TipoNoticia,
on_delete=models.CASCADE)

class Ubicacion(models.Model):
    id_ubicacion = models.AutoField(primary_key=True)
    nombre = models.CharField(max_length=255, unique=True)
    latitud = models.DecimalField(max_digits=9, decimal_places=6)
    longitud = models.DecimalField(max_digits=9, decimal_places=6)

class NoticiaUbicacion(models.Model):
    id_noticia = models.ForeignKey(Noticia,
on_delete=models.CASCADE)
    id_ubicacion = models.ForeignKey(Ubicacion,
on_delete=models.CASCADE)

    class Meta:
        constraints = [
models.UniqueConstraint(fields=['id_noticia', 'id_ubicacion'], name=
'unique_noticia_ubicacion')
]
```

3.6.3 Flujo de Almacenamiento

El proceso de almacenamiento sigue los siguientes pasos:

1. **Filtrado de noticias duplicadas:** Se realiza mediante la función `filtrar_noticias_existentes_por_urls()`, que consulta a la API de Django si las URLs ya están registradas:

```
response =
requests.post("http://django:8000/api/news/filtro_urls",
json={"urls": batch})
```

2. **Extracción de ubicaciones y clasificación:** Las noticias no duplicadas son pasadas por `extract_news_locations()` para extraer entidades y sus coordenadas.
3. **Almacenamiento en base de datos:** se utiliza la función `save_to_db()` que realiza una petición POST a la API REST del backend para almacenar tanto la noticia como las ubicaciones asociadas:

```
response =
requests.post("http://django:8000/api/news/create",
json=payload)
```

Dentro de `save_to_db()`, se construye un **payload** completo con todos los campos necesarios:

```
{
  "noticia": {
    "title": "Accidente en Colectora",
    "text": "Una colisión entre dos vehículos...",
    "date": "2024-04-01",
    "url": "https://ejemplo.com/accidente",
    "highlight_quote": "Uno de los conductores resultó
herido",
    "type": 1,
    "portal": "el-eco"
  },
  "ubicacion_validas": [
    {
      "nombre": "Colectora Sur J. C. Pugliese 123",
      "coordenadas": [-37.320435, -59.103363]
    }
  ]
}
```

3.6.4 Consideraciones Adicionales

- Las URLs de las APIs están actualmente fijadas estáticamente como `http://django:8000/...`. Para entornos de desarrollo, esto es aceptable, pero en producción debería utilizarse una **variable de entorno** configurable (por ejemplo, `DJANGO_API_URL`) para mayor flexibilidad y portabilidad.
- Se utilizó el módulo **requests** para hacer llamadas HTTP desde los scripts de integración, y se registran logs con el estado de cada inserción.

3.6.5 Ventajas del Uso de Django

El uso de Django como backend ofrece múltiples ventajas que favorecen el desarrollo y mantenimiento del sistema. En primer lugar, permite definir la estructura de la base de datos directamente desde el lenguaje Python, lo que simplifica la representación de entidades y sus relaciones. Además, el sistema genera automáticamente las migraciones necesarias para reflejar los cambios en la estructura de datos, evitando errores manuales y asegurando consistencia entre el código y la base de datos. Django también proporciona validaciones automáticas sobre los modelos definidos, lo que reduce la probabilidad de almacenar información incompleta o inconsistente. Por otra parte, gracias a Django REST Framework, se facilita considerablemente el desarrollo de APIs RESTful, lo cual resulta clave para la integración entre los distintos módulos del sistema. En conjunto, este enfoque permite centralizar la lógica de persistencia de datos en un backend robusto, extensible y mantenible.

3.7 Exposición de Datos (Backend del Sistema)

El backend del sistema fue desarrollado utilizando el framework **Django**, junto con **Django REST Framework** (DRF) para la creación de APIs RESTful. Esta elección permitió construir una arquitectura robusta, modular y fácil de mantener, en línea con las necesidades del proyecto.

3.7.1 Arquitectura General

El backend está organizado en la carpeta ``backend/app/'`, donde se encuentra la configuración principal del proyecto y la aplicación llamada **core**. Esta aplicación contiene los modelos, vistas, serializadores y endpoints de la API. La arquitectura fue diseñada con separación de responsabilidades, permitiendo un desarrollo ordenado y escalable.

Estructura principal:

```
backend/  
└─ app/  
    └─ core/  
        ├── models.py  
        ├── views.py  
        ├── api.py  
        ├── serializers.py  
        ├── urls.py  
        └─ ...  
    ├── manage.py  
    └─ django_docker/  
        ├── settings.py  
        └─ ...
```

3.7.2 Modelos y Persistencia de Datos

El sistema cuenta con cuatro modelos principales que estructuran el almacenamiento de noticias y sus relaciones con categorías y ubicaciones geográficas. Estos modelos fueron definidos en `models.py` y se sincronizan con la base de datos mediante el ORM de Django.

Los modelos son:

- **TipoNoticia:** Almacena categorías como "Accidente Vial", "Robo", etc.
- **Noticia:** Representa una noticia con campos como título, fecha, URL, tipo, texto, etc.
- **Ubicacion:** Contiene el nombre de la calle o intersección y las coordenadas geográficas (latitud y longitud).
- **NoticiaUbicacion:** Relación intermedia que vincula noticias con múltiples ubicaciones.

Ejemplo de definición de modelos:

```
class Noticia(models.Model):
    id_noticia = models.AutoField(primary_key=True)
    titulo = models.TextField()
    url = models.TextField(unique=True)
    fecha = models.DateField()
    texto = models.TextField()
    portal = models.CharField(max_length=30)
    id_tipo = models.ForeignKey(TipoNoticia,
                                on_delete=models.CASCADE)
```

Gracias al uso del ORM, se evita la escritura manual de SQL, se obtiene validación automática y se pueden generar migraciones con facilidad.

3.7.3 Serializadores

Los serializadores definen cómo se convierten los modelos a formatos JSON y viceversa. Se implementaron tanto para la visualización (**NoticiaSerializer**, **UbicacionSerializer**, etc.) como para la carga de datos (**NoticiaInputSerializer**).

Por ejemplo, **NoticiaInputSerializer** permite validar el esquema de carga que llega desde el módulo de scraping:

```
class NoticiaInputSerializer(serializers.Serializer):
    noticia = serializers.DictField()
    ubicacion_validas = UbicacionInputSerializer(many=True)
```

Esto asegura que los datos que llegan al backend respeten una estructura predefinida.

3.7.4 Endpoints y Lógica de Almacenamiento

La API expone tres funcionalidades principales:

1. **Carga de noticias nuevas** (POST `/api/news/create`):
Usa la clase `NoticiaCreateAPIView`, donde:

- Se recibe el título, contenido, fecha, tipo y ubicación.
- Se crea la noticia solo si no existe previamente (verificación por URL).
- Se crea la ubicación solo si no está registrada.
- Se vinculan las noticias con las ubicaciones mediante la tabla intermedia.

Esta operación usa `get_or_create()` para evitar duplicaciones.

2. **Filtrado de noticias** (GET `/api/news/`):

Permite consultar noticias por tipo, rango de fechas y por ubicación geográfica (radio en km). Las ubicaciones válidas se filtran por cercanía mediante el cálculo de distancia haversine.

3. **Filtro de URLs no registradas** (POST `/api/news/filtro_urls/`):

Devuelve aquellas URLs que aún no existen en la base de datos, optimizando así el almacenamiento al evitar duplicados.

3.7.5 Configuración y Despliegue con Docker

El sistema fue diseñado para funcionar en un entorno Dockerizado, lo que facilita su despliegue en diferentes entornos. Se utilizaron los siguientes contenedores:

- **db**: Contenedor de PostgreSQL.
- **backend**: Contenedor de Django.
- **frontend**: Interfaz web.
- **cron**: Módulo encargado de ejecutar procesos periódicos.

Ejemplo de parte del `docker-compose.yml`:

```
backend:
  build: ./docker/backend
  container_name: django
  command: ${DJANGO_COMMAND}
  volumes:
    - ./backend:/backend
  ports:
    - "8000:8000"
  environment:
    - POSTGRES_DB=${POSTGRES_DB}
    - POSTGRES_USER=${POSTGRES_USER}
    - POSTGRES_PASSWORD=${POSTGRES_PASSWORD}
  env_file:
    - .env
  depends_on:
    - db
```

La conexión con la base de datos se encuentra configurada en el archivo `settings.py`, utilizando variables de entorno definidas en `.env`, tal como se explicó en la sección “Almacenamiento de Datos en la Base de Datos”. Esta configuración permite desacoplar los datos sensibles del código fuente y facilita el despliegue en diferentes entornos (desarrollo, producción, etc.).

La implementación del backend con Django y Django REST Framework demostró ser altamente beneficiosa a lo largo del desarrollo. La posibilidad de definir modelos desde Python, gestionar la persistencia de forma declarativa mediante el ORM, y construir APIs REST de manera estructurada y segura, permitió mantener una arquitectura limpia, coherente y fácil de escalar. Esta elección, complementada con el uso de Docker y variables de entorno, contribuyó a una integración fluida con los demás componentes del sistema y a su vez facilita el despliegue en diferentes entornos de ejecución.

3.8 Visualización de Datos (Frontend del Sistema)

El frontend del sistema fue desarrollado utilizando **Next.js**, un framework moderno basado en React que permite construir aplicaciones web de tipo Single Page Application (SPA) con excelente rendimiento tanto en desarrollo como en producción. La elección de Next.js se debió a su soporte nativo para TypeScript, renderizado híbrido (SSR y CSR), buena integración con API REST y una comunidad muy activa.

Este módulo tiene como objetivo principal ofrecer una interfaz para que el usuario final pueda visualizar las noticias clasificadas y geolocalizadas. La interfaz permite aplicar filtros (tipo de noticia, fecha y ubicación) y ver los resultados tanto en un mapa como en una lista interactiva.

3.8.1 Estructura del Proyecto

El código fuente del frontend se encuentra organizado bajo la carpeta `frontend/`, y está organizado según las convenciones de Next.js:

```
frontend/
├── public/                # Archivos estáticos (iconos, imágenes
de marcadores, etc.)
├── src/
│   └── app/
│       ├── components/   # Componentes reutilizables como Map,
Sidebar, NewsList
│       ├── lib/          # Funciones auxiliares
│       ├── services/     # Lógica de consumo de APIs
│       ├── types.tsx     # Tipos TypeScript compartidos
│       └── page.tsx      # Página principal
├── next.config.ts        # Configuración de Next.js
├── tsconfig.json         # Configuración de TypeScript
└── package.json
```

3.8.2 Componentes Principales

- **Sidebar**: componente ubicado a la izquierda, permite al usuario aplicar filtros de búsqueda. Se especifican el tipo de evento, el rango de fechas y una ubicación con radio de distancia.
- **Map**: componente que renderiza el mapa interactivo utilizando la librería **Leaflet**, mostrando los eventos sobre un mapa mediante marcadores personalizados.

- **NewsList:** componente que lista las noticias devueltas por el sistema. Permite seleccionar una para centrar el mapa en su ubicación.

La lógica del mapa incluye normalización de datos, agrupamiento por coordenadas (para mostrar múltiples eventos en un mismo punto) y control inteligente del centrado del mapa dependiendo de si hay interacción del usuario.

3.8.3 Consumo de la API del Backend

La comunicación con el backend se realiza mediante **peticiones HTTP**. Las consultas se envían desde el componente `Sidebar.tsx` utilizando `fetch`. Por ejemplo:

```
const applyFilters = useCallback(async () => {
    const selectedTypes =
Object.keys(filters.newsTypes).filter((tipo) =>
filters.newsTypes[tipo]);
    const currentLocation = searchedLocation || userLocation;

    const queryParams = new URLSearchParams();
    selectedTypes.forEach((tipo) => queryParams.append('tipo',
tipo));
    if (filters.dateFrom) queryParams.append('fecha_desde',
filters.dateFrom);
    if (filters.dateTo) queryParams.append('fecha_hasta',
filters.dateTo);

    if (filters.range && currentLocation) {
        const rangeValue = parseFloat(filters.range);
        if (rangeValue > 0) {
            queryParams.append('distancia', rangeValue.toString());
            queryParams.append('lat', currentLocation[0].toString());
            queryParams.append('lon', currentLocation[1].toString());
        }
    }

    const res = await fetch(
        `${process.env.NEXT_PUBLIC_BACKEND_URL}/api/news/?${queryPara
ms.toString()}`
    );
    const data: NewsItem[] = await res.json();
    onConfirm(data);
}, [filters, searchedLocation, userLocation, onConfirm]);
```

Este esquema permite enviar filtros como tipo de evento, fecha y coordenadas geográficas con radio de búsqueda, y recibir una lista de noticias compatibles con el formato esperado por los componentes.

Inicialmente se utilizaba una URL fija (`http://localhost:8000`), pero fue reemplazada por una **variable de entorno** (`NEXT_PUBLIC_BACKEND_URL`) para facilitar la portabilidad entre entornos de desarrollo y producción.

3.8.4 Variables de Entorno en Next.js

Las variables de entorno deben comenzar con el prefijo `NEXT_PUBLIC_` para estar disponibles en el navegador. En el archivo `.env` del frontend se definió:

```
NEXT_PUBLIC_BACKEND_URL=http://localhost:8000
```

Esto garantiza que el valor pueda ser accedido desde cualquier componente del cliente, como se muestra en el ejemplo anterior.

3.8.5 Despliegue con Docker

El frontend se distribuye como un contenedor Docker, integrado con el resto del sistema mediante `docker compose`. La configuración del servicio en el archivo `docker-compose.yml` incluye:

```
frontend:
  build:
    context: .
    dockerfile: docker/frontend/Dockerfile
  ports:
    - "3000:3000"
  volumes:
    - ./frontend:/app
    - /app/node_modules
  environment:
    - NODE_ENV=development
  depends_on:
    - backend
```

El contenedor ejecuta Next.js en el puerto 3000 y se conecta al backend mediante la URL definida en el entorno.

El contenedor del frontend se construye utilizando el archivo `Dockerfile` ubicado en `docker/frontend/`, el cual define el entorno a partir de una imagen ligera de Node.js (`node:22.14-alpine`). En él se copian los archivos de configuración del proyecto, se instalan las dependencias necesarias —incluyendo Leaflet y sus tipos para TypeScript—, y se expone el puerto 3000 para acceder a la aplicación. El comando por defecto ejecuta `npm run dev`, iniciando el servidor de desarrollo de Next.js.

Ejemplo del `Dockerfile` utilizado:

```
FROM node:22.14-alpine
WORKDIR /app
```

```

COPY ../../frontend/package*.json ./
RUN npm install \
    leaflet \
    react-leaflet \
    lucide-react \
    --save-dev @types/leaflet
COPY ../../frontend ./
EXPOSE 3000
CMD ["npm", "run", "dev"]

```

3.8.6 Estilo Visual

El diseño de la interfaz se desarrolló con **Tailwind CSS**, lo que permitió construir componentes responsivos, modernos y visualmente consistentes sin necesidad de escribir CSS personalizado en gran medida. El uso de **TypeScript** ayudó a reforzar la integridad de datos, en especial al manipular respuestas dinámicas de la API REST.

Para la visualización geográfica, se utilizó **Leaflet**, una librería de código abierto especializada en mapas interactivos. El componente `Map.tsx` maneja la representación de los eventos sobre el mapa, utilizando marcadores personalizados provistos por Leaflet. Cada marcador representa uno o más eventos, y en caso de coincidencia de coordenadas, se agrupan. Para lograr esta agrupación, se implementó una función que organiza las noticias por coordenadas geográficas únicas utilizando `useMemo`. Esto evita recomputaciones innecesarias y mejora la eficiencia cuando el número de eventos es alto:

```

const groupedLocations = useMemo(() => {
  if (!normalizedNews.length) return [];
  const grouped = new Map<string, { location: Location; news:
  NewsItem[] }>();
  normalizedNews.forEach((item) => {
    item.locations.forEach((loc) => {
      const key = `${loc.coordinates[0]},${loc.coordinates[1]}`;
      if (!grouped.has(key)) grouped.set(key, { location: loc,
      news: [] });
      grouped.get(key)!.news.push(item);
    });
  });
  return Array.from(grouped.values());
}, [normalizedNews]);

```

Cuando el usuario interactúa con el mapa (zoom o arrastre), se desactiva el centrado automático para mejorar la experiencia de navegación.

Los íconos se asignan de acuerdo a la categoría del evento (robo, accidente, incendio, asesinato, etc.). En caso de que una ubicación contenga múltiples tipos de eventos, se utiliza un ícono genérico azul para representar la mezcla.

Para esto se tiene definido de la siguiente manera en `Map.tsx`:

```
const CATEGORY_ICONS = {
  robo: new L.Icon({ iconUrl: '/icons/robbery.png', iconSize: [25, 25] }),
  incendio: new L.Icon({ iconUrl: '/icons/fuego.png', iconSize: [25, 25] }),
  asesinato: new L.Icon({ iconUrl: '/icons/muerte.png', iconSize: [25, 25] }),
  'accidente vial': new L.Icon({ iconUrl: '/icons/accident.png', iconSize: [35, 35] }),
  default: new L.Icon({ iconUrl: '/marker-blue.png', iconSize: [25, 41] })
};
```

Donde se utiliza de la siguiente manera:

```
{groupedLocations.map(({ location, news }) => {
  const isMixedCategories = hasMultipleCategories(news);
  return (
    <Marker
      key={`-${location.id}-${news[0]?.id || 'default'}`}
      position={location.coordinates}
      icon={
        isMixedCategories
          ? CATEGORY_ICONS.default
          : CATEGORY_ICONS[news[0]?.category as keyof typeof CATEGORY_ICONS] ||
            CATEGORY_ICONS.default
      }
      eventHandlers={{ click: handleMarkerClick }}
    >
      <Popup className={styles['leaflet-popup-custom']}>
        <NewsPopup news={news} />
      </Popup>
    </Marker>
  );
})}
```

El componente `NewsPopup` encapsula la visualización de los detalles de una o más noticias asociadas a un mismo marcador. Recibe como prop una lista de noticias, y dentro del `Popup` de Leaflet muestra su título, fecha y enlace a la fuente. Esto permite una interacción clara y contextual directamente sobre el mapa, sin necesidad de navegar a otra sección.

El componente también renderiza la posición actual del usuario y la ubicación seleccionada en la búsqueda, mejorando la interacción entre el mapa y los filtros seleccionados.

3.8.7 Optimización del Renderizado

Para mejorar el rendimiento general del sistema y asegurar una experiencia fluida, se aplicaron técnicas de optimización propias de React:

- **`useMemo`**: utilizado para memorizar resultados costosos como la agrupación de ubicaciones (`groupedLocations`) o la transformación de noticias (`normalizedNews`), evitando cálculos innecesarios en cada render.
- **`useCallback`**: usado para definir funciones estables como `applyFilters` o `hasMultipleCategories`, evitando que se redefinan en cada render.
- **`memo`**: empleado para evitar que el componente principal `Map.tsx` se vuelva a renderizar cuando sus props no han cambiado.

Estas estrategias contribuyen a un frontend eficiente, especialmente al manejar grandes volúmenes de noticias y ubicaciones simultáneamente.

4. Casos de Estudio

Esta sección tiene como objetivo demostrar el funcionamiento práctico del sistema en escenarios reales, destacando su capacidad para clasificar noticias, geolocalizar eventos y ofrecer una experiencia personalizada al usuario. A continuación, se presentan casos concretos que ilustran el rendimiento del sistema, sus aciertos y sus limitaciones.

4.1 Caso de Estudio 1: Accidente Vial Detectado Correctamente y Procesado con Éxito

Este caso ejemplifica el funcionamiento integral del sistema ante una noticia real, desde su recolección automatizada hasta su correcta visualización en la interfaz. Se seleccionó una noticia que cumplió todas las etapas previstas del proceso, permitiendo evaluar la eficacia del sistema en un flujo completo de procesamiento.

4.1.1 Fuente de Datos

Como parte del scraping diario, el sistema extrajo automáticamente una noticia publicada por el portal **ABC Hoy**, correspondiente al día **07/04/2025**. En esta etapa inicial, la noticia no contaba aún con los campos `type` (clasificación) ni `ubicacion_validas` (entidades geográficas), ya que esos atributos se generan posteriormente mediante procesamiento automatizado.

4.1.2 Noticia Extraída

```
[
  {
    "indice": 3,
    "noticia": {
      "title": "Una moto chocó con una ciclista y se dio a la fuga",
      "url": "https://www.abchoy.com.ar/leernota/203011/una_moto_choco_con_una_ciclista_y_se_dio_a_la_fuga.html",
      "date": "07/04/2025",
      "highlight_quote": "El hecho ocurrió este lunes y la joven ciclista debió ser trasladada al Hospital Municipal Ramón Santamarina.",
      "text": "Un nuevo incidente vial se registró en la ciudad en la mañana de este lunes, alrededor de las 8, cuando una motocicleta colisionó con una ciclista en la intersección de las calles Franklin e Italia. Según testigos presenciales, el conductor de la moto, de color rojo y blanco, se dio a la fuga inmediatamente después del impacto, sin prestar asistencia a la víctima. Al lugar arribó una ambulancia del SAME quienes asistieron a la víctima y luego la trasladaron al Hospital Municipal Ramón Santamarina.",
      "portal": "abc"
    }
  }
]
```

```
}
]
```

4.1.3 Clasificación Automática

A través de su modelo de clasificación, el sistema analizó el contenido de la noticia y le asignó el tipo **1**, correspondiente a **accidente vial**. Esta categoría forma parte de las consideradas relevantes, por lo que se habilitó el procesamiento posterior.

```
[
{
  "indice": 3,
  "type": 1,
  "noticia": { ... }
}
```

4.1.4 Extracción de Entidades y Geolocalización

A partir del texto completo, el sistema detectó una mención explícita a una **intersección de calles**: *Franklin e Italia*. Utilizando su componente de extracción de entidades, construyó una consulta basada en esa referencia y la envió a un servicio externo de geolocalización.

La respuesta fue satisfactoria, retornando una ubicación concreta dentro del mapa de la ciudad de Tandil:

- **Entidad reconocida:** Franklin e Italia
- **Coordenadas:** -37.3166133, -59.1131998

Este proceso implicó identificar la entidad textual, construir una consulta basada en la misma, y enviarla al servicio de geolocalización, que respondió exitosamente con una ubicación concreta en el mapa de Tandil.

```
[
{
  "indice": 3,
  "type": 1,
  "noticia": { ... },
  "ubicacion_validas": [
    {
      "coordenadas": [
        -37.3166133,
        -59.1131998
      ],
      "nombre": "Franklin Y Italia"
    }
  ]
}
```

```
}
]
```

4.1.5 Resultado Final

Gracias al procesamiento exitoso, esta noticia fue incorporada al conjunto de eventos relevantes accesibles desde la interfaz del sistema. La ubicación fue validada y georreferenciada correctamente, permitiendo además calcular su **distancia relativa al usuario** al momento de aplicar filtros personalizados.

En la interfaz final, la noticia es visible con su título, fecha, tipo (accidente vial) y un marcador en el mapa sobre la esquina Franklin e Italia, facilitando una consulta clara y georreferenciada del hecho.

4.1.6 Análisis

Este caso demuestra que, en condiciones ideales, el sistema logra procesar noticias automáticamente desde su extracción hasta su visualización, pasando por todas las etapas propuestas:

- Scraping exitoso.
- Clasificación automática precisa.
- Extracción y validación de una entidad geográfica.
- Geolocalización correcta.
- Inclusión en la interfaz final como evento relevante.

Refleja además la capacidad del sistema para operar de forma **completamente automática**, dejando al usuario únicamente la tarea de explorar y filtrar noticias relevantes según sus intereses o ubicación.

4.2 Caso de Estudio 2: Clasificación Incorrecta de un Caso de Abuso como "Asesinato"

Durante la evaluación del sistema de clasificación automática de noticias policiales, se identificó un error relevante que permite analizar los límites del modelo actual. Se trató de una noticia clasificada erróneamente bajo la categoría "**asesinato**" (4), cuando el contenido real correspondía con mayor precisión a la categoría "**otro**" (0).

4.2.1 Detalles del Caso

La noticia, publicada en el portal El Eco, describe un juicio oral en el que se acusa a tres hombres de haber sometido sexualmente a una joven con discapacidades físicas y cognitivas, en un contexto de abuso reiterado y vulnerabilidad ([El Eco, 2025](#)). Según el artículo, los acusados habrían intercambiado relaciones sexuales no consentidas por dinero o bienes materiales. La imputación legal se centró en la figura de "promoción o facilitación de la prostitución de mayores de edad agravada" y no en un homicidio ni tentativa del mismo.

4.2.2 Análisis del Error

Este caso fue erróneamente clasificado como **"asesinato"**, lo cual pone en evidencia dos posibles causas del error:

1. **Presencia de lenguaje connotativamente fuerte:** El modelo puede haber interpretado ciertas expresiones intensas o emocionales (como "sometieron", "la usaron", "la cosificaron", "la desecharon") como indicios de violencia letal, cuando en realidad se referían a abusos sexuales y explotación, sin evidencia de muerte.
2. **Ausencia de una categoría específica para abuso sexual:** Dado que la categoría de clasificación más cercana disponible en el modelo es "asesinato", es probable que haya intentado forzar la etiqueta hacia el hecho violento más grave del conjunto, al no contar con una clase representativa para delitos sexuales.

4.2.3 Conclusión del Caso

Este caso muestra que, en sistemas automáticos de clasificación de noticias basados en aprendizaje automático, la definición y cobertura adecuada de las **clases de salida** es tan importante como el entrenamiento del modelo en sí. En este caso, la falta de una etiqueta como **"abuso sexual" o "delito contra la integridad sexual"** puede generar ambigüedades y errores significativos. Incorporar nuevas clases o reestructurar las ya existentes podría mejorar sustancialmente la precisión del modelo, especialmente en noticias con contenido sensible y complejo.

4.3 Caso de Estudio 3: Falla en la Geolocalización por Falta de Normalización de Entidad

Este caso ejemplifica un fallo ocurrido no en la etapa de clasificación automática, sino en el proceso de **extracción y validación de entidades geográficas**, debido a una discrepancia entre cómo las personas nombran informalmente las calles y cómo están registradas formalmente en los datos del sistema.

4.3.1 Detalles del Caso

El sistema clasificó correctamente una noticia como **accidente vial** (tipo 1), publicada el **04/04/2025** en el portal *ABC Hoy*. El artículo relataba un choque entre dos vehículos en la intersección de **Pratt y Cabral**, que dejó como saldo un menor de 8 años herido y hospitalizado.

Fragmento relevante del texto:

“Un choque entre dos vehículos se produjo en la tarde de este viernes en la intersección de las calles Pratt y Cabral, dejando como saldo un menor de 8 años de edad con heridas leves.”

La noticia completa fue almacenada con la siguiente estructura:

```
{
```

```

"indice": 36,
"type": 1,
"noticia": {
  "title": "Un menor hospitalizado tras un choque en Pratt y Cabral",
  "url": "https://www.abchoy.com.ar/leernota/202989/un_menor_hospitalizado_tras_un_choque_en_pratt_y_cabral.html",
  "date": "04/04/2025",
  ...
}
}

```

4.3.2 Problema Detectado

Durante la etapa de extracción y validación de entidades, el sistema identificó la intersección “Pratt y Cabral” como una posible mención geográfica. Sin embargo, al aplicar el algoritmo de **Approximate String Matching (ASM)**, se descartó la calle **Cabral** por no alcanzar el umbral mínimo de similitud requerido frente a los nombres presentes en el archivo ‘calles_preprocesado.txt’.

Aunque “**Pratt**” se encontraba correctamente en dicho archivo, “**Cabral**” no coincidía con ninguna entrada válida debido a que su forma registrada es “**José A. Cabral**”. Dado que la forma incompleta “Cabral” no superó el umbral de similitud definido, el sistema consideró inválida la entidad antes incluso de consultar la API de geolocalización.

Mensaje de error generado:

```
Invalid location: Intersección: Pratt y Cabral
```

5.3.3 Solución Propuesta

Para resolver este tipo de fallos de forma escalable, se propuso una mejora en la etapa de normalización de entidades: incorporar al archivo ‘calles_preprocesado.txt’ las formas comunes abreviadas junto a su forma formal, mediante una estructura del tipo:

```
cabral, jose a. cabral
```

De este modo, cuando el sistema detecte una mención parcial (como “Cabral”), podrá reemplazarla automáticamente por su denominación completa antes de consultar la API de geolocalización. Esto **reduce la tasa de errores** y permite reconocer intersecciones que en la práctica son mencionadas de forma coloquial, como lo haría un ciudadano local.

4.3.4 Conclusión del Caso

Este caso evidencia la importancia de incorporar **conocimiento local y contextos lingüísticos reales** en el diseño de sistemas de procesamiento automático. A pesar de que el clasificador y la extracción de entidad funcionaron correctamente, el fallo en la normalización evitó que la noticia fuera

georreferenciada correctamente. Pequeñas adaptaciones en los datos de soporte —como el diccionario de calles— pueden mejorar sustancialmente la precisión del sistema y su utilidad práctica.

5. Conclusión

El desarrollo de este proyecto integrador permitió aplicar de manera práctica una amplia gama de conocimientos adquiridos a lo largo de la carrera de Ingeniería en Sistemas. Se logró diseñar e implementar un sistema completo para la recolección, clasificación, geolocalización y visualización de noticias policiales locales, utilizando técnicas modernas de procesamiento de lenguaje natural, aprendizaje automático y desarrollo web.

El sistema fue capaz de recorrer múltiples etapas del procesamiento de datos, desde el scraping de noticias en portales locales hasta la clasificación semántica de su contenido y la extracción de ubicaciones geográficas. Además, se construyó una interfaz amigable donde los usuarios pueden consultar los hechos ordenados por cercanía e interés.

Este proyecto representa un caso real de aplicación de herramientas, tecnologías y enfoques que, si bien fueron estudiados en contextos académicos, se integraron aquí para construir una solución funcional que responde a una necesidad concreta: facilitar el acceso a información georreferenciada sobre hechos policiales. Los objetivos propuestos fueron alcanzados en su mayor parte, sentando una base sólida para futuras mejoras y despliegue.

5.1 Desarrollo del Trabajo

El proyecto se desarrolló de forma individual, pero con una coordinación activa y constante junto al director, mediante reuniones periódicas, correos electrónicos y la validación progresiva de cada etapa. A lo largo del proceso, se documentaron decisiones técnicas, pruebas, avances y dificultades, lo que permitió mantener una línea de trabajo ordenada y evaluable.

Para llevar adelante el desarrollo, fue necesario investigar documentación técnica, explorar herramientas específicas como Django, Next.js, Spacy, Nominatim y Docker, y estudiar alternativas de implementación para cada componente. Este aprendizaje autodidacta se complementó con los conocimientos adquiridos en distintas materias de la carrera, entre ellas:

- **Taller de Inteligencia Artificial**, que brindó las bases para entrenar modelos supervisados y realizar clasificación textual.
- **Base de Datos I y II**, fundamentales para el diseño del almacenamiento.
- **Diseño de Sistemas de Software y Metodología de Desarrollo**, claves para modularizar el código y gestionar etapas del proyecto.
- **Sistemas Operativos, Arquitectura de Computadoras y Comunicación de Datos**, que ayudaron a comprender cómo se comunican los procesos entre servicios y cómo optimizar su rendimiento.

La organización del trabajo se realizó por fases: diseño de arquitectura, implementación de scraping, entrenamiento del clasificador, extracción de entidades, integración con la API de geolocalización, desarrollo del backend, frontend y finalmente la interfaz visual. Este enfoque iterativo permitió avanzar asegurando el correcto funcionamiento de cada módulo antes de continuar con el siguiente.

5.2 Limitaciones

Si bien el sistema desarrollado demuestra un funcionamiento exitoso en entornos controlados y locales, aún presenta algunas limitaciones que deben ser consideradas para futuras mejoras:

- **Entorno de ejecución local:** Actualmente, el sistema completo (scraper, backend, frontend y base de datos) funciona mediante contenedores Docker en un entorno local. Esto limita su acceso únicamente al equipo donde se encuentra en ejecución, impidiendo su disponibilidad desde internet para otros usuarios.
- **Cobertura limitada en la extracción y normalización de entidades:** Aunque se implementó un sistema de detección y validación de entidades geográficas basado en coincidencia aproximada, existen casos donde no se logra una correcta identificación de calles o intersecciones debido a ambigüedades, abreviaciones o variaciones informales en la forma de nombrarlas. Esta situación impacta directamente en la precisión del módulo de geolocalización.
- **Clasificación sujeta a errores semánticos:** A pesar de que el modelo de clasificación logra un buen rendimiento general, algunos errores se deben a la ausencia de categorías más específicas (por ejemplo, abuso sexual), lo que puede llevar a asignaciones incorrectas en noticias complejas o ambiguas.
- **Dependencia de la estructura de los sitios web:** El proceso de scraping está ajustado a la estructura actual de los portales de noticias utilizados. Si alguno de estos sitios modifica su código HTML, la extracción de campos clave (como el texto principal o las URLs) puede fallar sin emitir errores explícitos, generando entradas incompletas o vacías. Actualmente, el sistema no cuenta con un mecanismo automatizado de detección de cambios estructurales en las páginas fuente. Esto implica que cualquier modificación en la web requiere una revisión manual del scraper correspondiente para adaptarlo.

5.3 Trabajos Futuros

Como continuación natural de este proyecto, se proponen distintas líneas de mejora que permitirían ampliar su funcionalidad, robustez y alcance real:

- **Despliegue del sistema en producción:** Una de las principales tareas a futuro consiste en implementar la infraestructura necesaria para que el sistema pueda ser accedido de forma pública. Esto podría lograrse desplegando el frontend y backend en un servidor web (como una instancia de AWS EC2, VPS, o plataforma como Render o Railway), utilizando herramientas como Nginx para servir la aplicación, y asegurando que la base de datos y el scheduler del scraper permanezcan en ejecución continua. Con ello, el sistema podría estar disponible en línea y ofrecer su funcionalidad a múltiples usuarios simultáneamente.
- **Mejora de la detección de entidades geográficas mediante IA:** Actualmente, el sistema utiliza expresiones regulares y aproximación léxica para extraer ubicaciones, lo cual presenta limitaciones ante casos complejos. Como línea de mejora, se podría integrar el uso de APIs de Named Entity Recognition (NER) basadas en inteligencia artificial (como spaCy con modelos más entrenados, HuggingFace o incluso APIs externas como OpenAI o IBM Watson),

para lograr una extracción más precisa de localidades y direcciones mencionadas en las noticias.

- **Ampliación y refinamiento de las categorías de clasificación:** Incorporar nuevas etiquetas específicas, como “abuso sexual” o “violencia de género”, permitiría una clasificación más precisa y sensible a la naturaleza de los hechos reportados. Además, un reentrenamiento del modelo con ejemplos adicionales y balanceados mejoraría su capacidad de generalización frente a textos variados.
- **Monitoreo automático del scraper frente a cambios en las fuentes:** Se podría desarrollar un sistema de verificación automática de la integridad del scraping, basado en métricas como el porcentaje de noticias con cuerpo vacío, URLs rotas o frecuencia inusualmente baja de resultados. Esto permitiría detectar a tiempo modificaciones en los portales fuente y emitir alertas o reportes para facilitar su mantenimiento.
- **Interfaz para personalización del usuario:** A futuro, se podría desarrollar una sección donde los usuarios registren sus preferencias (tipo de noticias, rango de distancia, zonas de interés), y así recibir resultados personalizados y ordenados por relevancia. Esto mejoraría la experiencia de uso y abriría la posibilidad de notificaciones o filtros avanzados.

6. Referencias

- Cortes, C., & Vapnik, V. (1995). *Support-vector networks*. *Machine learning*, 20, 273-297. <https://doi.org/10.1007/BF00994018>.
- Ramos, J. (2003, December). Using tf-idf to determine word relevance in document queries. In *Proceedings of the first instructional conference on machine learning* (Vol. 242, No. 1, pp. 29-48).
- Yang, Z., Yu, J., & Kitsuregawa, M. (2010, July). Fast algorithms for top-k approximate string matching. In *Proceedings of the AAAI conference on artificial intelligence* (Vol. 24, No. 1, pp. 1467-1473). <https://doi.org/10.1609/aaai.v24i1.7527>.
- PostgreSQL Global Development Group. (2024). *PostgreSQL documentation* (versión 16). <https://www.postgresql.org/docs/16/index.html>.
- Django Software Foundation. (2025). *Django documentation* (versión 5.2). <https://docs.djangoproject.com/en/5.2/>.
- Vercel. (2025). *Next.js documentation*. <https://nextjs.org/docs>.
- Zandbergen, P. A. (2008). A comparison of address point, parcel and street geocoding techniques. *Computers, Environment and Urban Systems*, 32(3), 214–232. <https://doi.org/10.1016/j.compenvurbsys.2007.11.006>.
- Glez-Peña, D., Lourenço, A., López-Fernández, H., Reboiro-Jato, M., & Fdez-Riverola, F. (2014). *Web scraping technologies in an API world*. *Briefings in bioinformatics*, 15(5), 788-797. <https://doi.org/10.1093/bib/bbt026>.
- Scrapy. (s.f.). *An open source and collaborative framework for extracting the data you need from websites*. Recuperado el 10 de diciembre de 2024, de <https://scrapy.org/>.
- Zhou, Z.-H. (2021). *Machine learning*. Springer Nature. <https://books.google.com.ar/books?id=ctM-EAAAQBAJ>.
- Bishop, C. M., & Nasrabadi, N. M. (2006). *Pattern recognition and machine learning* (Vol. 4, No. 4, p. 738). New York: springer. <https://link.springer.com/book/9780387310732>.
- Google Developers. (s.f.). *Geocoding API overview*. Google. Recuperado el 20 de marzo de 2025. <https://developers.google.com/maps/documentation/geocoding>.
- OpenStreetMap. (s.f.). *About OpenStreetMap – Copyright and License*. Recuperado el 20 de marzo de 2025. <https://www.openstreetmap.org/copyright>.
- Explosion. (s.f.). *spaCy: Industrial-strength Natural Language Processing in Python*. Recuperado el 20 de marzo de 2025. <https://spacy.io/>.
- Docker Inc. (2025). *Docker Documentation*. <https://docs.docker.com/>

Kibriya, A. M., Frank, E., Pfahringer, B., & Holmes, G. (2004, December). Multinomial naive bayes for text categorization revisited. In *Australasian joint conference on artificial intelligence* (pp. 488-499). Berlin, Heidelberg: Springer Berlin Heidelberg.

Hosmer Jr, D. W., Lemeshow, S., & Sturdivant, R. X. (2013). *Applied logistic regression*. John Wiley & Sons. <https://books.google.com.ar/books?id=bRoxQBIZRd4C>

Wallach, H. M. (2006, June). Topic modeling: beyond bag-of-words. In *Proceedings of the 23rd international conference on Machine learning* (pp. 977-984).

El Eco. (2025). *Pidieron penas de ocho a cinco años para el trío acusado de someter sexualmente a una joven discapacitada*. Recuperado el 5 de mayo de 2025, de <https://www.eleco.com.ar/policiales/pidieron-penas-de-ocho-a-cinco-anos-para-el-trio-acusado-de-someter-sexualmente-a-una-joven-discapacitada>.